



Give the heavy skeleton of part D, also explain how to do the remaining part too

Checking existing headers to align the skeleton with the codebase.

Let me read the pieces Part D has to plug into — your existing `exec` path, the command/job data types, the shell state, and the main loop — so the skeleton compiles against your actual code and calls your real functions rather than inventing new ones.

Ran 2 commands

This is the real IIT-H CS3.301 spec, so I've mapped the skeleton onto your actual `d_jobs.c` structure and the `exec_*` API it builds on. Let me pull the mechanical helper bodies (job table, async-safe printing, name building) so I can hand those to you complete and leave only the Part-D learning objectives as your work.

Ran 2 commands

Last piece — the foreground dispatch and the D1 sequencer, so the skeleton's control flow and my explanation of "command not found stops the sequence" (D1 rule 3) are exactly right.

Ran a command

I've now traced the whole module — the mechanical helpers, the `exec_*` /builtin APIs it plugs into, and exactly where the Part-D concepts live. Here's the heavy skeleton. Everything mechanical is written and compiles clean under your flags; the five spots that are the Part-D learning are left as clearly-specified `TODO`s.

Ran 2 commands, created a file

One expected hiccup: `wstr` / `wuint` are the async-safe printers you'll call in `TODO-1`, so with that `TODO` unfilled they read as "defined but unused" under `-Werror`. I'll mark them so the skeleton builds cleanly now and they're ready for you to use.

Edited a file, ran a command

Builds clean under `-Werror` and links; the binary runs (foreground sequencing is intentionally

+ Write a message...

Here's how to finish each `TODO`. Do them in this order — each one makes the next testable — and note the whole thing hinges on one idea: **a pipeline is one job, which is one process group.**

1. TODO-3 — process groups (do this first). Every stage of a pipeline joins a single group whose id (`pgid`) is the first stage's pid. Each child calls `setpgid` to join, and the parent calls it again for the same child; you do it on both sides because after `fork()` you don't control whether parent or child runs first, and the child might even `exec` before the parent gets there — whichever call lands first wins, the other is a harmless no-op. Once groups exist, the group-`wait` (`-pgid`) and the terminal handoff (`tcsetpgrp`) work, and background jobs stop stealing the terminal (spec #12). Test: run `sleep 50 &`, then in another terminal `ps -o pid,pgid,cmd` — the `sleep`'s `pgid` should equal its own pid and differ from the shell's.

2. TODO-2 — wait_group (foreground wait). Loop, waiting on the whole group (negative `pgid`) with `WUNTRACED`, counting down the alive members until they're all gone. Two traps: treat `errno == EINTR` as "a handler ran, just retry" rather than an error; and wait on the *group*, not a single pid — the classic bug is waiting on one pid so a 3-stage pipe only waits for one stage and the prompt races the output. Test: `echo hi | cat | cat` must print `hi` and only *then* prompt; `sleep 3` should freeze the prompt for 3 seconds.

3. TODO-1 — handle_dead (background completion message). This runs *inside* the `SIGCHLD` handler, so it must be async-signal-safe: build the message with the provided `wstr/wuint`, never `printf/fprintf` (calling those from a handler is the single most common crash in this part). Only print when the *last* member of a pipeline dies — that's what `nalive` tracks — choose "normally" vs "abnormally" from whether it was killed by a signal, and report the `pgid` as the pid (spec #13). Test: `sleep 2 &`, then just sit there — about 2s later you should see `sleep with pid <n> exited normally` appear even at an idle prompt (#10). `sleep 100 &` then `kill -9 <pid>` → `exited abnormally`.

4. TODO-4 — the [job] pid line. In the background branch, print the bracketed job number and pid before returning, and *don't* wait. Test: `sleep 5 &` prints `[1] <pid>` and gives you a prompt instantly; launch several and the numbers only ever climb, never repeat (#4).

5. TODO-5 — D1 rule 3. Before launching a single external command, resolve it with `exec_resolve`; if it can't be resolved, print the not-found line and return `RUN_FAILED`, which is what makes `jobs_run_sequence` stop the rest of the `;` chain. The boundary to get right: a command that *does* start and then exits non-zero has *not* failed (rule 4) — the sequence continues — so gate only on "couldn't start," never on exit status. Test: `echo a ; nope ; echo b` prints `a`, the not-found line, and stops (no `b`); `false ; echo b` still prints `b`.

Two things already handled in the plumbing that are worth understanding rather than rewriting: the `sigchld_block/unblock` around `launch_pipeline` is what defers a background completion message until a running foreground job finishes (spec #11) — that's why the block is there. And the `FG_STOPPED` branch plus the empty `jobs_activities/jobs_resume/jobs_ping`  are the seams where **Part E** (Ctrl-Z, `fg/bg`, `activities`, `ping`) drops in without re-cturing anything.

C-shell project skeleton code

2. TODO-2 — `wait_group` (foreground wait). Loop, waiting on the whole group (negative `pgid`) with `WUNTRACED`, counting down the alive members until they're all gone. Two traps: treat `errno == EINTR` as "a handler ran, just retry" rather than an error; and wait on the `group`, not a single `pid` — the classic bug is waiting on one `pid` so a 3-stage pipe only waits for one stage and the prompt races the output. Test: `echo hi | cat | cat` must print `hi` and only then prompt; `sleep 3` should freeze the prompt for 3 seconds.

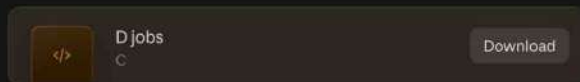
3. TODO-1 — `handle_dead` (background completion message). This runs *inside* the `SIGCHLD` handler, so it must be async-signal-safe: build the message with the provided `wstr / wuint`, never `printf / fprintf` (calling those from a handler is the single most common crash in this part). Only print when the *last* member of a pipeline dies — that's what `naive` tracks — choose "normally" vs "abnormally" from whether it was killed by a signal, and report the `pgid` as the `pid` (spec #13). Test: `sleep 2 &`, then just sit there — about 2s later you should see `sleep with pid <n> exited normally` appear even at an idle prompt (#10). `sleep 100 &` then `kill -9 <pid>` → `exited abnormally`.

4. TODO-4 — the `[job] pid` line. In the background branch, print the bracketed job number and `pid` before returning, and *don't* wait. Test: `sleep 5 &` prints `[1] <pid>` and gives you a prompt instantly; launch several and the numbers only ever climb, never repeat (#4).

5. TODO-5 — D1 rule 3. Before launching a single external command, resolve it with `exec_resolve`; if it can't be resolved, print the not-found line and return `RUN_FAILED`, which is what makes `jobs_run_sequence` stop the rest of the `;` chain. The boundary to get right: a command that *does* start and then exits non-zero has *not* failed (rule 4) — the sequence continues — so gate only on "couldn't start," never on exit status. Test: `echo a ; nope ; echo b` prints `a`, the not-found line, and stops (no `b`); `false ; echo b` still prints `b`.

Two things already handled in the plumbing that are worth understanding rather than rewriting: the `sigchld_block / unblock` around `launch_pipeline` is what defers a background completion message until a running foreground job finishes (spec #11) — that's why the block is there. And the `FG_STOPPED` branch plus the empty `jobs_activities / jobs_resume / jobs_ping` stubs are the seams where **Part E** (Ctrl-Z, `fg / bg`, `activities`, `ping`) drops in without restructuring anything.

Say "give the heavy skeleton of part E" (or F) and I'll build those the same way.



🔗 📄 📁 🔄 🗑️



+ Write a message...

🗣️ 🗣️

Claude is AI and can make mistakes. Please double-check responses.

Opus 4.8 High

D jobs · C

Copy 🗑️

```
24 #include "d_jobs.h"
25 #include "a_shell.h"
26 #include "b_builtins.h"
27 #include "c_exec.h"
28 #include "f_spy.h"
29 #include "f_snoop.h"
30
31 #include <errno.h>
32 #include <signal.h>
33 #include <stdio.h>
34 #include <stdlib.h>
35 #include <string.h>
36 #include <sys/wait.h>
37 #include <termios.h>
38 #include <unistd.h>
39
40 /* ---- Globals: the job table + a little shell state ----- */
41 static Job_t jobs[JOBS_MAX];
42 static int next_job_id = 1; /* monotonic: never reused (D2 #4) */
43 static pid_t shell_pgid = 0; /* the shell's own process group */
44 static int have_tty = 0; /* is stdin a real terminal? */
45
46 /* Result of waiting on a foreground pipeline. (Part E3 later adds a timeout
47  * variant; Part D only needs DONE vs STOPPED so a Ctrl-Z can't hang us.) */
48 typedef enum { FG_DONE, FG_STOPPED } FgResult;
49
50 /* ===== Table helpers (DONE) ===== */
51 static Job_t *job_alloc(void)
52 {
53     for (int i = 0; i < JOBS_MAX; i++) {
54         if (jobs[i].in_use == 0) {
55             Job_t *j = &jobs[i];
56             memset(j, 0, sizeof(*j));
57             j->in_use = 1;
58             j->job_id = next_job_id++;
59             j->status = JS_RUNNING;
60             return j;
61         }
62     }
63     return NULL;
64 }
65
66 static Job_t *job_find_by_pid(pid_t pid)
67 {
68     for (int i = 0; i < JOBS_MAX; i++) {
69         if (jobs[i].in_use == 0) continue;
70         for (int k = 0; k < jobs[i].npids; k++)
71             if (jobs[i].pids[k] == pid) return &jobs[i];
72     }
73 }
```


Copy

claude.ai

mini-project1/c-shell at main...xv6 — CS3.301C-shell project skeleton cod...https://pdos.csail.mit.edu/6.8...mit-pdos/xv6-riscv: Xv6 for...General Instructions — CS3....(5) Mod-01 Lec-17 TYPES O...

D jobs C

Copy

```
99  /* =====
100  * [TODO-1] handle_dead() - called from the SIGCHLD handler below
101  *
102  * Goal (D2 #8/#9/#10/#13): one pid of some pipeline just terminated.
103  * 1. Find its job with job_find_by_pid(pid). If NULL, return.
104  * 2. If it died from a signal ('signaled' != 0), mark j->abnormal = 1.
105  * 3. Decrement j->nalive (guard against going below 0).
106  * 4. When j->nalive == 0 the WHOLE pipeline is done, so print exactly:
107  *     <name> with pid <pgid> exited normally\n      (if !abnormal)
108  *     <name> with pid <pgid> exited abnormally\n      (if abnormal)
109  *     using wstr()/wvint() ONLY (async-safe). The reported pid is the
110  *     pipeline's pgid == first pid (D2 #13). Use j->name for <name>.
111  * 5. Free the slot: j->in_use = 0.
112  *
113  * Reminder: "exited normally" covers ANY WIFEXITED result - a non-zero
114  * exit status is still a normal exit (D2 #8).
115  * =====*/
116 static void handle_dead(pid_t pid, int signaled)
117 {
118     Job_t *j = job_find_by_pid(pid);
119     if (j == NULL) return;
120
121     /* TODO-1: mark abnormal, decrement nalive, and when it hits 0 print the
122      *     "<name> with pid <pgid> exited normally/abnormally" line with
123      *     wstr()/wvint(), then set j->in_use = 0. */
124     (void)signaled; /* remove once used */
125 }
126
127 /* ===== SIGCHLD handler =====
128  * D2 #6/#7: reap EVERY child that changed state, non-blocking, so the shell
129  * never blocks. WNOHANG => don't wait; loop until waitpid returns 0 / <0.
130  * WUNTRACED|WCONTINUED let us also notice bg jobs that stop/continue. */
131 static void on_sigchld(int sig)
132 {
133     (void)sig;
134     int saved = errno; /* a handler must not clobber errno */
135     int status;
136     pid_t pid;
137
138     while ((pid = waitpid(-1, &status, WNOHANG | WUNTRACED | WCONTINUED)) > 0) {
139         if (WIFEXITED(status) || WIFSIGNALED(status)) {
140             handle_dead(pid, WIFSIGNALED(status)); /* -> [TODO-1] */
141         } else if (WIFSTOPPED(status)) {
142             Job_t *j = job_find_by_pid(pid);
143             if (j) j->status = JS_STOPPED;
144         } else if (WIFCONTINUED(status)) {
145             Job_t *j = job_find_by_pid(pid);
146             if (j) j->status = JS_RUNNING;
147         }
148     }
```

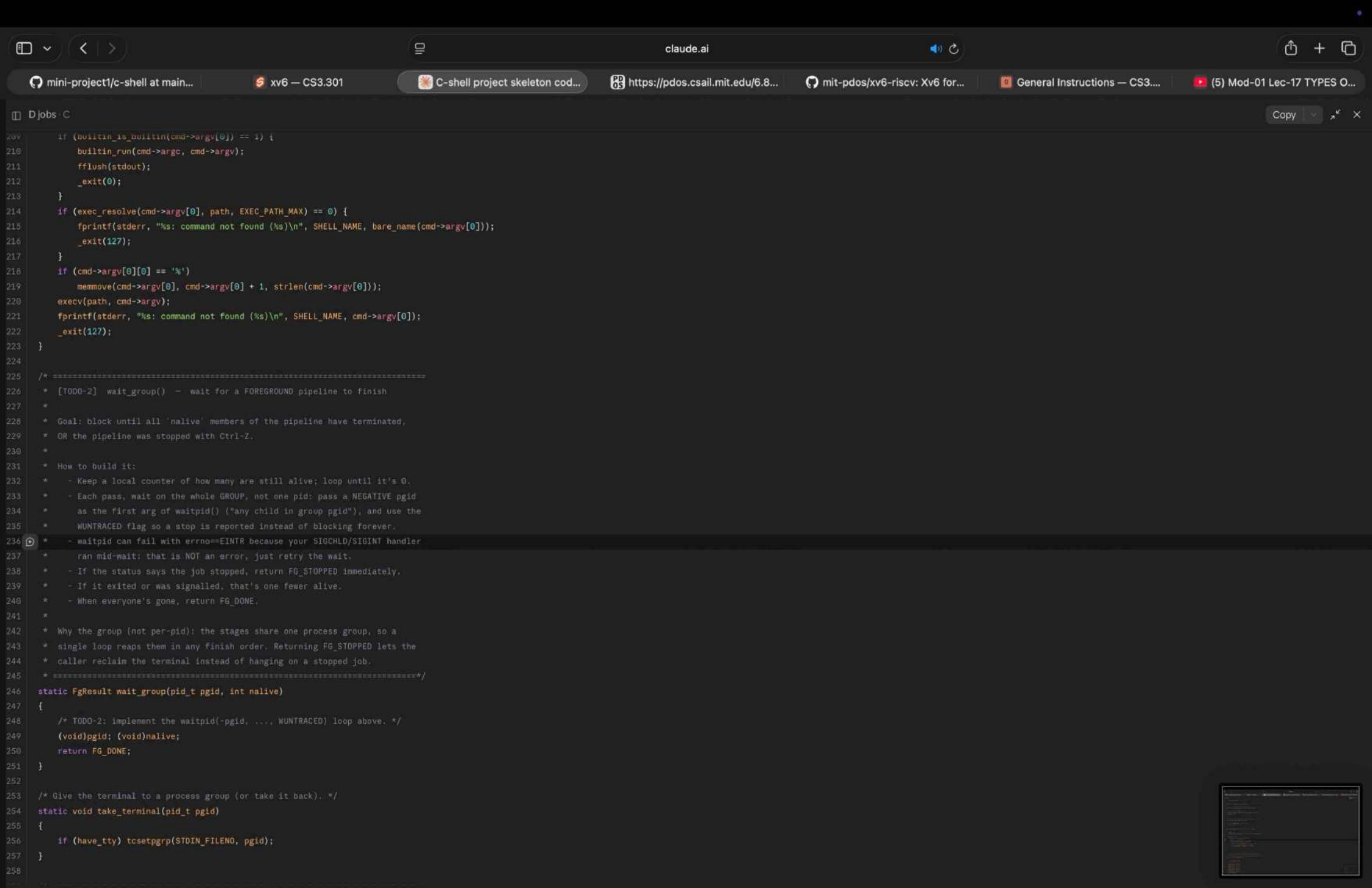
claude.ai

mini-project1/c-shell at main...xv6 — CS3.301C-shell project skeleton cod...https://pdos.csail.mit.edu/6.8...mit-pdos/xv6-riscv: Xv6 for...General Instructions — CS3....(5) Mod-01 Lec-17 TYPES O...

Djobs C

Copy

```
159     write(STDOUT_FILENO, "\n", 1);
160 }
161 static void on_sigstsp(int sig) { (void)sig; }
162
163 /* ===== Building display strings ===== */
164 static void str_append(char *dst, int cap, const char *src)
165 {
166     int len = (int)strlen(dst), i = 0;
167     while (src[i] != '\0' && len + 1 < cap) dst[len++] = src[i++];
168     dst[len] = '\0';
169 }
170
171 /* "%ls" is our "skip cwd, go to PATH" syntax; the *name* is still "ls". */
172 static const char *bare_name(const char *s)
173 {
174     if (s != NULL && s[0] == '%') return s + 1;
175     return (s != NULL) ? s : "";
176 }
177
178 static void build_names(Job *job, char *name, char *cmdline)
179 {
180     name[0] = '\0';
181     str_append(name, JOB_NAME_MAX, bare_name(job->commands[0]->argv[0]));
182
183     cmdline[0] = '\0';
184     for (int s = 0; s < job->command_count; s++) {
185         Command *c = job->commands[s];
186         if (s > 0) str_append(cmdline, JOB_NAME_MAX, " | ");
187         for (int a = 0; a < c->argc; a++) {
188             if (a > 0) str_append(cmdline, JOB_NAME_MAX, " ");
189             str_append(cmdline, JOB_NAME_MAX, c->argv[a]);
190         }
191     }
192 }
193
194 /* ===== Child side after fork =====
195  * A forked child must (a) restore default signal behaviour so Ctrl-C/Ctrl-Z
196  * actually reach it (the shell has these handled/ignored), then (b) exec. */
197 static void child_run(Command *cmd)
198 {
199     char path[EXEC_PATH_MAX];
200
201     signal(SIGINT, SIG_DFL);
202     signal(SIGTSTP, SIG_DFL);
203     signal(SIGTTOU, SIG_DFL);
204     signal(SIGTTIN, SIG_DFL);
205     signal(SIGQUIT, SIG_DFL);
206     signal(SIGCHLD, SIG_DFL);
207     sigchld_unblock();
208 }
```



claude.ai

mini-project1/c-shell at main...

xv6 — CS3.301

C-shell project skeleton cod...

https://pdos.csail.mit.edu/6.8...

mit-pdos/xv6-riscv: Xv6 for...

General Instructions — CS3...

(5) Mod-01 Lec-17 TYPES O...

Copy

D:\jobs C

```
253  /* Give the terminal to a process group (or take it back). */
254  static void take_terminal(pid_t pgid)
255  {
256      if (have_tty) tcsetpgrp(STDIN_FILENO, pgid);
257  }
258
259  /* =====
260   * launch_pipeline() - fork every stage of ONE pipeline, wired for pipes
261   *                    and redirection. The pipe/fork/dup2 plumbing is DONE.
262   *                    You fill [TODO-3] and [TODO-4].
263   * =====*/
264  static void launch_pipeline(Job *job, int background)
265  {
266      Redirection redirs[JOB_MAX_PIDS];
267      int  opened[JOB_MAX_PIDS];
268      int  stages  = job->command_count;
269      int  prev_read = -1;
270      pid_t pgid = 0;
271      pid_t plist[JOB_MAX_PIDS];
272      char pnm[JOB_MAX_PIDS][JOB_PNAME_MAX];
273      int  npids = 0;
274      char name[JOB_NAME_MAX], cmdline[JOB_NAME_MAX];
275
276      if (stages > JOB_MAX_PIDS) stages = JOB_MAX_PIDS;
277      build_names(job, name, cmdline);
278
279      sigchld_block(); /* keep the table stable while we build the job */
280
281      for (int i = 0; i < stages; i++) {
282          Command *cmd = job->commands[i];
283          int  pipe_fds[2];
284          int  has_pipe = 0;
285          pid_t pid;
286
287          opened[i] = 0;
288
289          if (i < stages - 1) { /* need a pipe to the next stage */
290              if (pipe(pipe_fds) < 0) { perror("cshell: pipe"); break; }
291              has_pipe = 1;
292          }
293          if (cmd->argc == 0 || redir_open(cmd, &redirs[i]) == 0) {
294              if (prev_read >= 0) close(prev_read);
295              prev_read = has_pipe ? (close(pipe_fds[1]), pipe_fds[0]) : -1;
296              continue;
297          }
298          opened[i] = 1;
299
300          fflush(stdout);
301          pid = fork();
302          if (pid < 0) { perror("cshell: fork"); break; }
```

claude.ai

mini-project1/c-shell at main...xv6 — CS3.301C-shell project skeleton cod...https://pdos.csail.mit.edu/6.8...mit-pdos/xv6-riscv: Xv6 for...General Instructions — CS3....(5) Mod-01 Lec-17 TYPES O...

Djobs C

Copy

```
303
304 if (pid == 0) { /* ----- CHILD ----- */
305     /* [TODO-3a] Put this child in the pipeline's process group with
306      * setpgid(). Pass 0 as the pid arg (means "me") and the group
307      * leader's pgid as the group. For the FIRST stage pgid is still
308      * 0, which conveniently makes that child its own group leader -
309      * which is what you want (pgid == first pid, D2 #13). */
310
311     if (prev_read >= 0) { /* stdin from previous pipe */
312         dup2(prev_read, STDIN_FILENO);
313         close(prev_read);
314     }
315     if (has_pipe) { /* stdout into next pipe */
316         close(pipe_fds[0]);
317         dup2(pipe_fds[1], STDOUT_FILENO);
318         close(pipe_fds[1]);
319     }
320     redir_apply(&redirs[i]); /* < > >> override pipes */
321     for (int t = 0; t < redirs[i].target_count; t++)
322         close(redirs[i].targets[t]);
323     child_run(cmd); /* never returns */
324 }
325
326 /* ----- PARENT ----- */
327 /* [TODO-3b] On the FIRST child, remember its pid as the group's pgid.
328  * Then call setpgid() from the PARENT too, to place this child in the
329  * group. Doing it on both sides is deliberate: after fork() you don't
330  * know whether parent or child runs first, and the child may even
331  * exec before you get here - whichever setpgid lands first wins, the
332  * other is a harmless no-op. Skip this and background jobs stay in the
333  * shell's group and fight for the terminal (breaks D2 #12). */
334
335 plist[npids] = pid;
336 strncpy(pnm[npids], bare_name(cmd->argv[0]), JOB_PNAME_MAX - 1);
337 pnm[npids][JOB_PNAME_MAX - 1] = '\0';
338 npids++;
339
340 if (prev_read >= 0) close(prev_read);
341 prev_read = has_pipe ? (close(pipe_fds[1]), pipe_fds[0]) : -1;
342 }
343 if (prev_read >= 0) close(prev_read);
344
345 /* ----- BACKGROUND ----- */
346 if (background == 1) {
347     Job_t *slot = job_alloc();
348     if (slot != NULL) {
349         slot->pgid = pgid;
350         slot->npids = npids;
351         slot->nalive = npids;
352         slot->status = JOB_RUNNING;
353     }
354 }
```

🏠 < >

claude.ai 🔊 ↻

📌 + 📄

🔍 mini-project1/c-shell at main...

🔍 xv6 — CS3.301

🔍 C-shell project skeleton cod...

🔍 https://pdos.csail.mit.edu/6.8...

🔍 mit-pdos/xv6-riscv: Xv6 for...

🔍 General Instructions — CS3....

🔍 (5) Mod-01 Lec-17 TYPES O...

D jobs C 🔍 Copy 🗑️

```
349     slot->pgid = pgid;
350     slot->npids = npids;
351     slot->nalive = npids;
352     slot->status = JS_RUNNING;
353     memcpy(slot->pids, plist, sizeof(pid_t) * (size_t)npids);
354     for (int i = 0; i < npids; i++) strcpy(slot->pnames[i], pnm[i]);
355     str_append(slot->name, JOB_NAME_MAX, name);
356     str_append(slot->cmdline, JOB_NAME_MAX, cmdline);
357
358     /* [TODO-4] Print the background job line here, BEFORE returning and
359     * before any command output (D2 #3). Format is bracketed job id,
360     * a space, then the pid - match the spec example "[1] 4021" - and
361     * the pid to report is the group's pgid (D2 #13). fflush() after.
362     * Then DO NOT wait: fall through, unblock SIGCHLD, return, so the
363     * shell prompts immediately (D2 #5). Completion gets reported
364     * later, asynchronously, by handle_dead() (D2 #10). */
365 }
366
367 for (int i = 0; i < stages; i++) if (opened[i]) redir_finish(&redirs[i]);
368 sigchld_unblock();
369 return;
370 }
371
372 /* ----- FOREGROUND ----- */
373 take_terminal(pgid); /* hand the tty to the job */
374 FgResult r = wait_group(pgid, npids); /* -> [TODO-2] */
375 take_terminal(shell_pgid); /* reclaim the tty */
376
377 for (int i = 0; i < stages; i++) if (opened[i]) redir_finish(&redirs[i]);
378
379 if (r == FG_STOPPED) {
380     /* Part E1 (later): a Ctrl-Z'd foreground job becomes a tracked
381     * STOPPED job here. For Part D you can leave this empty. */
382 }
383 sigchld_unblock();
384 }
385
386 /* ===== Foreground dispatch ===== */
387 #define RUN_OK 1
388 #define RUN_FAILED 0
389
390 /* A single builtin runs IN the shell process, so redirection is done by
391 * saving fd 0/1, dup2'ing, running, then restoring. */
392 static int run_builtin_with_redir(Command *cmd)
393 {
394     Redirection redir;
395     int saved_in = -1, saved_out = -1;
396
397     if (redir_open(cmd, &redir) == 0) return RUN_OK;
398     if (redir.in_fd >= 0) { saved_in = dup(STDIN_FILENO); dup2(redir.in_fd, STDIN_FILENO); }
399     if (redir.out_fd >= 0) { fflush(stdout); saved_out = dup(STDOUT_FILENO); dup2(redir.out_fd, STDOUT_FILENO); }
```

claude.ai

mini-project1/c-shell at main...xv6 — CS3.301C-shell project skeleton cod...https://pdos.csail.mit.edu/6.8...mit-pdos/xv6-riscv: Xv6 for...General Instructions — CS3...(5) Mod-01 Lec-17 TYPES O...

Djobs C

Copy

```
400     builtin_run(cmd->argc, cmd->argv);
401     fflush(stdout);
402
403     if (saved_in >= 0) { dup2(saved_in, STDIN_FILENO); close(saved_in); }
404     if (saved_out >= 0) { dup2(saved_out, STDOUT_FILENO); close(saved_out); }
405     redir_finish(&redir);
406     return RUN_OK;
407 }
408
409 /* Part E stubs - implement in Part E. Present so the file links now. */
410 static void jobs_activities(void) { /* Part E1 */ }
411 static void jobs_resume(int argc, char **a) { (void)argc; (void)a; /* Part E3 */ }
412 static void jobs_ping(int argc, char **a) { (void)argc; (void)a; /* Part E4 */ }
413
414 static int run_foreground(Job *job)
415 {
416     if (job->command_count == 1) {
417         Command *cmd = job->commands[0];
418         char path[EXEC_PATH_MAX];
419
420         if (cmd->argc == 0) return RUN_OK;
421
422         /* job-control + custom builtins run in the shell (they need the table) */
423         if (strcmp(cmd->argv[0], "activities") == 0) { jobs_activities(); return RUN_OK; }
424         if (strcmp(cmd->argv[0], "resume") == 0) { jobs_resume(cmd->argc, cmd->argv); return RUN_OK; }
425         if (strcmp(cmd->argv[0], "ping") == 0) { jobs_ping(cmd->argc, cmd->argv); return RUN_OK; }
426         if (strcmp(cmd->argv[0], "spy") == 0) { spy_run(cmd->argc, cmd->argv); return RUN_OK; }
427         if (strcmp(cmd->argv[0], "snoop") == 0) { snoop_run(cmd->argc, cmd->argv); return RUN_OK; }
428
429         if (builtin_is_builtin(cmd->argv[0]) == 1)
430             return run_builtin_with_redir(cmd);
431
432         /* [TODO-5] D1 rule 3: a single external command that CANNOT be started
433          * at all must NOT launch, and must STOP the ';' sequence. Try to
434          * resolve it into 'path' with exec_resolve() first; if that fails,
435          * print the standard "cshell: command not found (name)" line
436          * (SHELL_NAME + bare_name()) and return RUN_FAILED - that return is
437          * what makes jobs_run_sequence() break out of the sequence.
438          * Note the boundary: a command that DOES start and then exits with a
439          * non-zero status has NOT failed (D1 rule 4) - the sequence keeps
440          * going - so only the "can't resolve" case returns RUN_FAILED here. */
441         (void)path; /* remove once you use it in TODO-5 */
442     }
443     launch_pipeline(job, 0);
444     return RUN_OK;
445 }
446
447 /* ===== Public API ===== */
448 static void install(int signo, void (*handler)(int), int restart)
449 {
```

claude.ai

mini-project1/c-shell at main...

xv6 — CS3.301

C-shell project skeleton cod...

https://pdos.csail.mit.edu/6.8...

mit-pdos/xv6-riscv: Xv6 for...

General Instructions — CS3...

(5) Mod-01 Lec-17 TYPES O...

Copy

D:\jobs C

```
450     struct sigaction sa;
451     memset(&sa, 0, sizeof(sa));
452     sa.sa_handler = handler;
453     sigemptyset(&sa.sa_mask);
454     sa.sa_flags = restart ? SA_RESTART : 0;
455     sigaction(signo, &sa, NULL);
456 }
457
458 /* Call once at startup (already wired into a_main.c). Installs the SIGCHLD
459  * reaper (D2 #6) and puts the shell in charge of the terminal.      */
460 void jobs_init(void)
461 {
462     for (int i = 0; i < JOBS_MAX; i++) jobs[i].in_use = 0;
463
464     install(SIGCHLD, on_sigchld, 1); /* SA_RESTART so read() isn't broken */
465     install(SIGINT, on_sigint, 0); /* no restart => read() gets EINTR */
466     install(SIGTSTP, on_sigstp, 1); /* catch+ignore: shell won't stop */
467     signal(SIGTTOU, SIG_IGN);
468     signal(SIGTTIN, SIG_IGN);
469
470     have_tty = isatty(STDIN_FILENO);
471     setpgid(0, 0);
472     shell_pgid = getpgrp();
473     if (have_tty) tcsetpgrp(STDIN_FILENO, shell_pgid);
474 }
475
476 /* D1: run every job in the list left-to-right. Backgrounds are launched and
477  * not waited for; foregrounds are waited for; a foreground that cannot START
478  * stops the rest of the sequence (D1 rule 3).      */
479 void jobs_run_sequence(JobList *list, const char *raw_line)
480 {
481     (void)raw_line; /* Part E uses this to store cmdline */
482     for (int i = 0; i < list->job_count; i++) {
483         Job *job = list->jobs[i];
484         if (job->command_count == 0) continue;
485         if (job->command_count == 1 && job->commands[0]->argc == 0) continue;
486
487         if (job->background == 1) {
488             launch_pipeline(job, 1);
489         } else {
490             if (run_foreground(job) == RUN_FAILED)
491                 break; /* D1 rule 3 */
492         }
493     }
494 }
495
496 int jobs_has_stopped(void)
497 {
498     for (int i = 0; i < JOBS_MAX; i++)
499         if (jobs[i].in_use == 1 && jobs[i].status == JS_STOPPED)
```

claude.ai

mini-project1/c-shell at main...

xv6 — CS3.301

C-shell project skeleton cod...

https://pdos.csail.mit.edu/6.8...

mit-pdos/xv6-riscv: Xv6 for...

General Instructions — CS3...

(5) Mod-01 Lec-17 TYPES O...

Copy

D:\jobs C

```
463
464     install(SIGCHLD, on_sigchld, 1); /* SA_RESTART so read() isn't broken */
465     install(SIGINT, on_sigint, 0); /* no restart => read() gets EINTR */
466     install(SIGSTP, on_sigstp, 1); /* catch+ignore: shell won't stop */
467     signal(SIGTTOU, SIG_IGN);
468     signal(SIGTTIN, SIG_IGN);
469
470     have_tty = isatty(STDIN_FILENO);
471     setpgid(0, 0);
472     shell_pgid = getpgrp();
473     if (have_tty) tcsetpgrp(STDIN_FILENO, shell_pgid);
474 }
475
476 /* D1: run every job in the list left-to-right. Backgrounds are launched and
477  * not waited for; foregrounds are waited for; a foreground that cannot START
478  * stops the rest of the sequence (D1 rule 3). */
479 void jobs_run_sequence(JobList *list, const char *raw_line)
480 {
481     (void)raw_line; /* Part E uses this to store cmdline */
482     for (int i = 0; i < list->jobs_count; i++) {
483         Job *job = list->jobs[i];
484         if (job->command_count == 0) continue;
485         if (job->command_count == 1 && job->commands[0]->argc == 0) continue;
486
487         if (job->background == 1) {
488             launch_pipeline(job, 1);
489         } else {
490             if (run_foreground(job) == RUN_FAILED)
491                 break; /* D1 rule 3 */
492         }
493     }
494 }
495
496 int jobs_has_stopped(void)
497 {
498     for (int i = 0; i < JOBS_MAX; i++)
499         if (jobs[i].in_use == 1 && jobs[i].status == JS_STOPPED)
500             return 1;
501     return 0;
502 }
503
504 void jobs_hangup_all(void)
505 {
506     for (int i = 0; i < JOBS_MAX; i++) {
507         if (jobs[i].in_use == 0) continue;
508         kill(-jobs[i].pgid, SIGHUP);
509         kill(-jobs[i].pgid, SIGCONT); /* so a stopped group receives it */
510     }
511 }
512
```

Give the heavy skeleton for part E and then how to do the rest of it

Presented file >

Part E lives in the same `d_jobs.c`. The signal handlers, `tcsetpgrp` handoff, Ctrl-D handling, and `SIGHUP` -on-exit are already in place, so Part E comes down to five `1000` s. In implementation order:

1. TODO-E2 — record a Ctrl-Z'd job. When a foreground pipeline stops, it isn't in the job table yet, so the `FG_STOPPED` branch allocates a slot and copies the pids in — you set its status to `JS_STOPPED` and print `[id] + Stopped <cmdline>`. This relies on `wait_group` returning `FG_STOPPED` on `WIFSTOPPED`, so make sure that path handles stops, not just exits. Test: run `sleep 100`, press Ctrl-Z → `[1] + Stopped sleep 100`, back at the prompt, shell never stops.

2. TODO-E1 — the `activities` process lines. The oldest-first grouping and group header are written; you write the inner line per process. Probe each pid with `kill(pid, 0)` and skip it if that fails, so exited members aren't listed; otherwise print two-space-indented `<pid> <name> <state>` with state from the job's `status`. Test: `sleep 100 &`, then `cat | sort &`, then `activities` — grouped output with `Running / Stopped`.

3. TODO-E3 — `resume` actions. Parsing and error strings are done; the job-control half is yours. Send `SIGCONT` to `-pgid` and mark Running; for `bg`, print `[id] + Running <cmdline>` and return without touching the terminal; for `fg`, echo the command line, then `take_terminal → wait_group(pgid, nalive, timeout) → reclaim`, and handle the result (`FG_DONE` free the slot, `FG_STOPPED` re-stop and print, `FG_TIMEDOUT` free and print `resume: job timed out`). Two gotchas: **re-find the job by id after waiting** — `SIGCHLD` may have freed the slot while you blocked — and the `--timeout` detection lives in `wait_group`'s `EINTR` branch (arming is already wired: when `g_alarm` is set mid-wait, send the group `SIGTERM`, reap, return `FG_TIMEDOUT`). Test: Ctrl-Z a `sleep 100`, then `resume %1 bg`, `resume %1 fg`, and `resume %1 fg --timeout 3 → resume: job timed out`.

4. TODO-E4 — `ping` delivery. Signal validation and `% 64` are done; you resolve the target and send. `%n → job_find_by_id`, signal the whole group with `kill(-pgid, actual)`; a bare number → a pid, accepted only if `job_find_by_pid` shows this shell is tracking it, then `kill(pid, actual)`. Print `Sent signal <sig> to <target>` echoing the *original* number, not the modulo'd one; any unknown target or failed `kill` prints `ping: no such process found`. Ordering is already handled — signal validated before target lookup, and negatives fail `all_digits` so they're `invalid syntax` rather than reduced by the modulo. Test: `ping 4030 15`, `ping %2 9`, `ping 4030 79` (delivers 15, echoes 79), `ping 99999 9 → not found`, `ping 4030 abc` and `ping 4030 -3 → invalid syntax`. ↓

+ Write a message...

claude.ai

mini-project1/c-shell at main...

xv6 — CS3.301

C-shell project skeleton cod...

https://pdos.csail.mit.edu/6.8...

mit-pdos/xv6-riscv: Xv6 for...

Mini Project 1 — CS3.301

(5) Mod-01 Lec-17 TYPES O...

Djobs C

Copy

```
1  /*
2  * d_jobs.c - Part D (sequential ';' and background '&' execution) and
3  *           Part E (activities, terminal control, resume, ping).
4  *
5  * Implemented here: the job table and its helpers, async-signal-safe output
6  * helpers, the SIGCHLD/SIGINT/SIGTSTP/SIGALRM setup, the pipeline
7  * fork/pipe/redirect wiring, the foreground dispatch, the D1 sequencer, the
8  * terminal give/reclaim helpers, and the argument parsing / job lookups and
9  * fixed output strings for activities / resume / ping.
10 *
11 * Marked TODO (see the inline notes) - the D and E logic to fill in:
12 *   D  : completion reporting, foreground group-wait, process-group formation,
13 *       the "[job] pid" line, and the D1 not-found stop rule.
14 *   E1 : the per-process print inside activities.
15 *   E2 : recording a Ctrl-Z'd foreground pipeline as a stopped job.
16 *   E3 : SIGCONT + fg/bg terminal handling + the --timeout path in resume.
17 *   E4 : target resolution + signal delivery in ping.
18 *
19 * Compiles as-is under -std=c23 -Wall -Wextra -Werror; fill the TODOs to make
20 * it functional and drop the (void) placeholders as each is implemented.
21 */
22
23 #include "d_jobs.h"
24 #include "a_shell.h"
25 #include "b_builtins.h"
26 #include "c_exec.h"
27 #include "f_spy.h"
28 #include "f_snoop.h"
29
30 #include <errno.h>
31 #include <signal.h>
32 #include <stdio.h>
33 #include <stdlib.h>
34 #include <string.h>
35 #include <sys/wait.h>
36 #include <termios.h>
37 #include <unistd.h>
38
39 /* ---- Globals: the job table + a little shell state ----- */
40 static Job_t jobs[JOBS_MAX];
41 static int  next_job_id = 1; /* monotonic; never reused (D2 #4) */
42 static pid_t shell_pgid = 0; /* the shell's own process group */
43 static int  have_tty    = 0; /* is stdin a real terminal? */
44
45 static volatile sig_atomic_t g_alarm = 0; /* set by the SIGALRM handler (E3) */
46
47 /* Result of waiting on a foreground pipeline:
48 *   FG_DONE    - every member terminated
49 *   FG_STOPPED - the pipeline was stopped with Ctrl-Z (E2)
50 *   FG_TIMEOUT - a 'resume --timeout' deadline elapsed first (E3) */
```

Copy

□ D jobs C

claude.ai

mini-project1/c-shell at main...

xv6 — CS3.301

C-shell project skeleton cod...

https://pdos.csail.mit.edu/6.8...

mit-pdos/xv6-riscv: Xv6 for...

Mini Project 1 — CS3.301

(5) Mod-01 Lec-17 TYPES O...

Djobs C

Copy

✖

```
94  /* ===== Async-signal-safe printing =====
95  * printf() is NOT safe to call inside a signal handler, write() is. These
96  * two are what you use to print the completion message from handle_dead(). */
97  __attribute__((unused)) static void wstr(const char *s)
98  {
99      write(STDOUT_FILENO, s, strlen(s));
100  }
101
102  __attribute__((unused)) static void wuint(unsigned long v)
103  {
104      char buf[24];
105      int i = (int)sizeof(buf);
106      if (v == 0) { write(STDOUT_FILENO, "0", 1); return; }
107      while (v > 0 && i > 0) { buf[--i] = (char)('0' + (v % 10)); v /= 10; }
108      write(STDOUT_FILENO, buf + i, (size_t)((int)sizeof(buf) - i));
109  }
110
111  /* ---- Block SIGCHLD around code that touches the job table ----
112  * The SIGCHLD handler also touches the table, so we mask it while the main
113  * loop is mid-update to avoid a torn read/write. */
114  static void sigchld_block(void)
115  {
116      sigset_t s; sigemptyset(&s); sigaddset(&s, SIGCHLD);
117      sigprocmask(SIG_BLOCK, &s, NULL);
118  }
119  static void sigchld_unblock(void)
120  {
121      sigset_t s; sigemptyset(&s); sigaddset(&s, SIGCHLD);
122      sigprocmask(SIG_UNBLOCK, &s, NULL);
123  }
124
125  /* =====
126  * [TODO-1] handle_dead() - called from the SIGCHLD handler below
127  *
128  * Goal (D2 #8/#9/#10/#13): one pid of some pipeline just terminated.
129  * 1. Find its job with job_find_by_pid(pid). If NULL, return.
130  * 2. If it died from a signal ('signaled' != 0), mark j->abnormal = 1.
131  * 3. Decrement j->nalive (guard against going below 0).
132  * 4. When j->nalive == 0 the WHOLE pipeline is done, so print exactly:
133  *    <name> with pid <pgid> exited normally\n      (if !abnormal)
134  *    <name> with pid <pgid> exited abnormally\n    (if abnormal)
135  *    using wstr()/wuint() ONLY (async-safe). The reported pid is the
136  *    pipeline's pgid == first pid (D2 #13). Use j->name for <name>.
137  * 5. Free the slot: j->in_use = 0.
138  *
139  * Reminder: "exited normally" covers ANY WIFEXITED result - a non-zero
140  * exit status is still a normal exit (D2 #8).
141  * =====*/
142  static void handle_dead(pid_t pid, int signaled)
143  {
```

claude.ai

mini-project1/c-shell at main...xv6 — CS3.301C-shell project skeleton cod...https://pdos.csail.mit.edu/6.8...mit-pdos/xv6-riscv: Xv6 for...Mini Project 1 — CS3.301(5) Mod-01 Lec-17 TYPES O...

Djobs C

Copy

```
144     Job_t *j = job_find_by_pid(pid);
145     if (j == NULL) return;
146
147     /* TODO-1: mark abnormal, decrement nalive, and when it hits 0 print the
148      *    "<name> with pid <pgid> exited normally/abnormally" line with
149      *    wstr()/wint(), then set j->in_use = 0. */
150     (void)signaled; /* remove once used */
151 }
152
153 /* ===== SIGCHLD handler =====
154  * D2 #6/#7: reap EVERY child that changed state, non-blocking, so the shell
155  * never blocks. WNOHANG => don't wait; loop until waitpid returns 0 / <0.
156  * WUNTRACED|WCONTINUED let us also notice bg jobs that stop/continue. */
157 static void on_sigchld(int sig)
158 {
159     (void)sig;
160     int saved = errno; /* a handler must not clobber errno */
161     int status;
162     pid_t pid;
163
164     while ((pid = waitpid(-1, &status, WNOHANG | WUNTRACED | WCONTINUED)) > 0) {
165         if (WIFEXITED(status) || WIFSIGNALED(status)) {
166             handle_dead(pid, WIFSIGNALED(status)); /* -> [TODO-1] */
167         } else if (WIFSTOPPED(status)) {
168             Job_t *j = job_find_by_pid(pid);
169             if (j) j->status = JS_STOPPED;
170         } else if (WIFCONTINUED(status)) {
171             Job_t *j = job_find_by_pid(pid);
172             if (j) j->status = JS_RUNNING;
173         }
174     }
175     errno = saved;
176 }
177
178 /* ---- Prompt-level signal handlers ----
179  * Ctrl-C: the shell itself must survive; drop the half-typed line.
180  * Ctrl-Z: the shell must NOT stop, so we catch-and-ignore at the prompt. */
181 static void on_sigint(int sig)
182 {
183     (void)sig;
184     shell_sigint_flag = 1;
185     write(STDOUT_FILENO, "\n", 1);
186 }
187 static void on_sigstsp(int sig) { (void)sig; }
188
189 /* E3: the 'resume --timeout <e>' deadline. The handler only flips a flag; the
190  * wait loop checks it and does the actual SIGTERM + cleanup. */
191 static void on_sigalrm(int sig) { (void)sig; g_alarm = 1; }
192
193 /* ===== Building display strings ===== */
194
```

claude.ai

mini-project1/c-shell at main...

xv6 — CS3.301

C-shell project skeleton cod...

https://pdos.csail.mit.edu/6.8...

mit-pdos/xv6-riscv: Xv6 for...

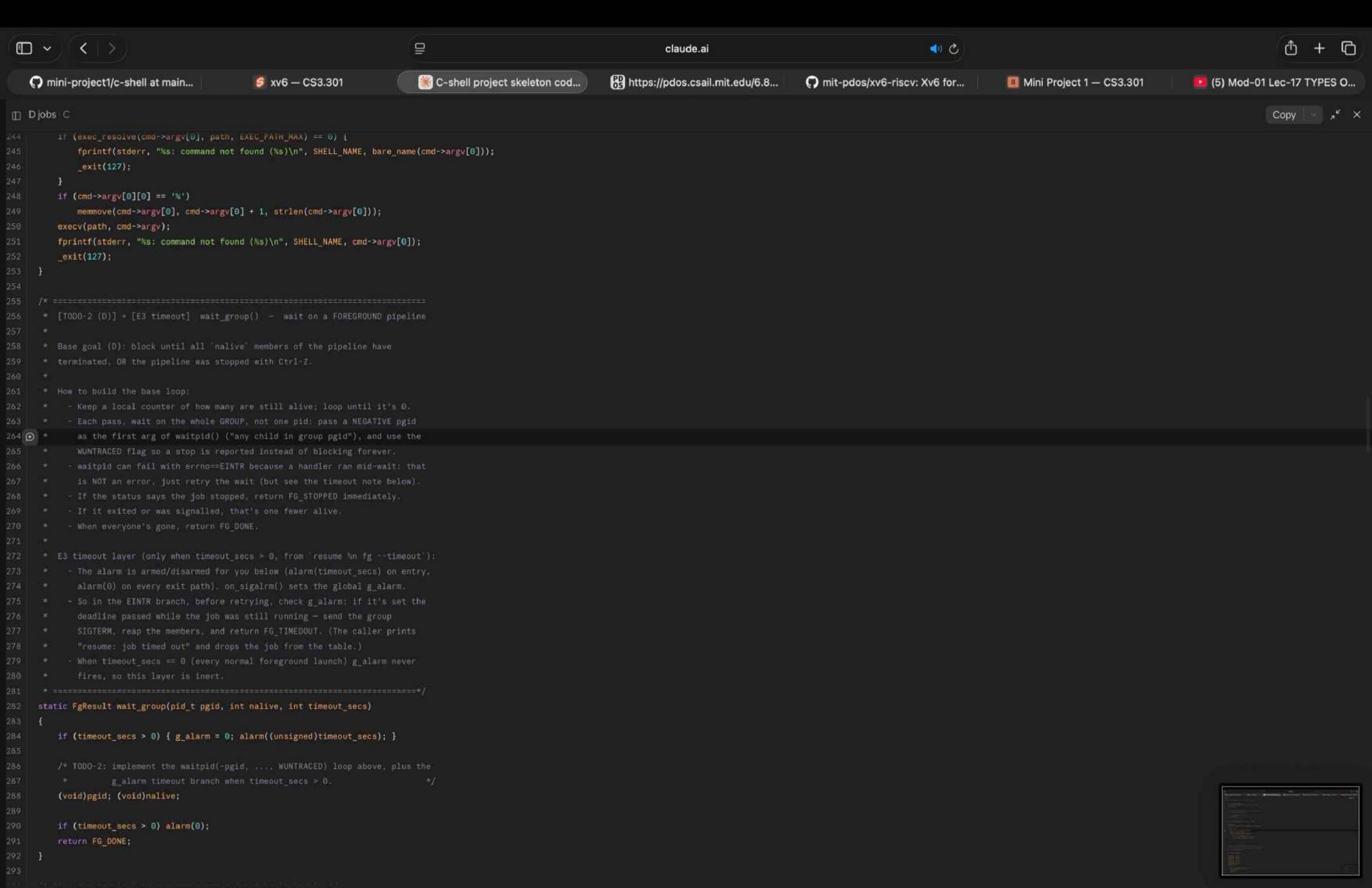
Mini Project 1 — CS3.301

(5) Mod-01 Lec-17 TYPES O...

Copy

D:\jobs C

```
194 static void str_append(char *dst, int cap, const char *src)
195 {
196     int len = (int)strlen(dst), i = 0;
197     while (src[i] != '\0' && len + 1 < cap) dst[len++] = src[i++];
198     dst[len] = '\0';
199 }
200
201 /* "%ls" is our "skip cmd, go to PATH" syntax; the *name* is still "ls". */
202 static const char *bare_name(const char *s)
203 {
204     if (s != NULL && s[0] == '%') return s + 1;
205     return (s != NULL) ? s : "";
206 }
207
208 static void build_names(Job *job, char *name, char *cmdline)
209 {
210     name[0] = '\0';
211     str_append(name, JOB_NAME_MAX, bare_name(job->commands[0]->argv[0]));
212
213     cmdline[0] = '\0';
214     for (int s = 0; s < job->command_count; s++) {
215         Command *c = job->commands[s];
216         if (s > 0) str_append(cmdline, JOB_NAME_MAX, " | ");
217         for (int a = 0; a < c->argc; a++) {
218             if (a > 0) str_append(cmdline, JOB_NAME_MAX, " ");
219             str_append(cmdline, JOB_NAME_MAX, c->argv[a]);
220         }
221     }
222 }
223
224 /* ===== Child side after fork =====
225  * A forked child must (a) restore default signal behaviour so Ctrl-C/Ctrl-Z
226  * actually reach it (the shell has these handled/ignored), then (b) exec. */
227 static void child_run(Command *cmd)
228 {
229     char path[EXEC_PATH_MAX];
230
231     signal(SIGINT, SIG_DFL);
232     signal(SIGTSTP, SIG_DFL);
233     signal(SIGTTOU, SIG_DFL);
234     signal(SIGTTIN, SIG_DFL);
235     signal(SIGQUIT, SIG_DFL);
236     signal(SIGCHLD, SIG_DFL);
237     sigchld_unblock();
238
239     if (builtin_is_builtin(cmd->argv[0]) == 1) {
240         builtin_run(cmd->argv, cmd->argv);
241         fflush(stdout);
242         _exit(0);
243     }
```



claude.ai

mini-project1/c-shell at main...xv6 — CS3.301C-shell project skeleton cod...https://pdos.csail.mit.edu/6.8...mit-pdos/xv6-riscv: Xv6 for...Mini Project 1 — CS3.301(5) Mod-01 Lec-17 TYPES O...

Djobs C

Copy

```
294 /* Give the terminal to a process group (or take it back). */
295 static void take_terminal(pid_t pgid)
296 {
297     if (have_tty) tcsetpgrp(STDIN_FILENO, pgid);
298 }
299
300 /* =====
301  * launch_pipeline() - fork every stage of ONE pipeline, wired for pipes
302  *                    and redirection. The pipe/fork/dup2 plumbing is DONE.
303  *                    You fill [TODO-3] and [TODO-4].
304  * =====*/
305 static void launch_pipeline(Job *job, int background)
306 {
307     Redirection redirs[JOB_MAX_PIDS];
308     int opened[JOB_MAX_PIDS];
309     int stages = job->command_count;
310     int prev_read = -1;
311     pid_t pgid = 0;
312     pid_t plist[JOB_MAX_PIDS];
313     char pnm[JOB_MAX_PIDS][JOB_PNAME_MAX];
314     int npids = 0;
315     char name[JOB_NAME_MAX], cmdline[JOB_NAME_MAX];
316
317     if (stages > JOB_MAX_PIDS) stages = JOB_MAX_PIDS;
318     build_names(job, name, cmdline);
319
320     sigchld_block(); /* keep the table stable while we build the job */
321
322     for (int i = 0; i < stages; i++) {
323         Command *cmd = job->commands[i];
324         int pipe_fds[2];
325         int has_pipe = 0;
326         pid_t pid;
327
328         opened[i] = 0;
329
330         if (i < stages - 1) { /* need a pipe to the next stage */
331             if (pipe(pipe_fds) < 0) { perror("cshell: pipe"); break; }
332             has_pipe = 1;
333         }
334         if (cmd->argc == 0 || redir_open(cmd, &redirs[i]) == 0) {
335             if (prev_read >= 0) close(prev_read);
336             prev_read = has_pipe ? (close(pipe_fds[1]), pipe_fds[0]) : -1;
337             continue;
338         }
339         opened[i] = 1;
340
341         fflush(stdout);
342         pid = fork();
343         if (pid < 0) { perror("cshell: fork"); break; }
344     }
345 }
```

```

344
345     if (pid == 0) {                /* ----- CHILD ----- */
346         /* [TODO-3a] Put this child in the pipeline's process group with
347          * setpgid(). Pass 0 as the pid arg (means "me") and the group
348          * leader's pgid as the group. For the FIRST stage pgid is still
349          * 0, which conveniently makes that child its own group leader -
350          * which is what you want (pgid == first pid, D2 #13). */
351
352         if (prev_read >= 0) {      /* stdin from previous pipe */
353             dup2(prev_read, STDIN_FILENO);
354             close(prev_read);
355         }
356         if (has_pipe) {            /* stdout into next pipe */
357             close(pipe_fds[0]);
358             dup2(pipe_fds[1], STDOUT_FILENO);
359             close(pipe_fds[1]);
360         }
361         redir_apply(&redirs[i]);    /* < > >> override pipes */
362         for (int t = 0; t < redirs[i].target_count; t++)
363             close(redirs[i].targets[t]);
364         child_run(cmd);             /* never returns */
365     }
366
367     /* ----- PARENT ----- */
368     /* [TODO-3b] On the FIRST child, remember its pid as the group's pgid.
369      * Then call setpgid() from the PARENT too, to place this child in the
370      * group. Doing it on both sides is deliberate: after fork() you don't
371      * know whether parent or child runs first, and the child may even
372      * exec before you get here - whichever setpgid lands first wins, the
373      * other is a harmless no-op. Skip this and background jobs stay in the
374      * shell's group and fight for the terminal (breaks D2 #12). */
375
376     plist[npids] = pid;
377     strncpy(pnm[npids], bare_name(cmd->argv[0]), JOB_PNAME_MAX - 1);
378     pnm[npids][JOB_PNAME_MAX - 1] = '\0';
379     npids++;
380
381     if (prev_read >= 0) close(prev_read);
382     prev_read = has_pipe ? (close(pipe_fds[1]), pipe_fds[0]) : -1;
383 }
384 if (prev_read >= 0) close(prev_read);
385
386 /* ----- BACKGROUND ----- */
387 if (background == 1) {
388     Job_t *slot = job_alloc();
389     if (slot != NULL) {
390         slot->pgid = pgid;
391         slot->npids = npids;
392         slot->nalive = npids;
393         slot->status = JS_RUNNING;

```

claude.ai

mini-project1/c-shell at main...xv6 — CS3.301C-shell project skeleton cod...https://pdos.csail.mit.edu/6.8...mit-pdos/xv6-riscv: Xv6 for...Mini Project 1 — CS3.301(5) Mod-01 Lec-17 TYPES O...

D jobs C

Copy

```
393     slot->status = JS_RUNNING;
394     memcpy(slot->pids, plist, sizeof(pid_t) * (size_t)npids);
395     for (int i = 0; i < npids; i++) strcpy(slot->pnames[i], pnm[i]);
396     str_append(slot->name, JOB_NAME_MAX, name);
397     str_append(slot->cmdline, JOB_NAME_MAX, cmdline);
398
399     /* [TODO-4] Print the background job line here, BEFORE returning and
400      * before any command output (D2 #3). Format is bracketed job id,
401      * a space, then the pid - match the spec example "[1] 4021" - and
402      * the pid to report is the group's pgid (D2 #13). fflush() after.
403      * Then DO NOT wait: fall through, unblock SIGCHLD, return, so the
404      * shell prompts immediately (D2 #5). Completion gets reported
405      * later, asynchronously, by handle_dead() (D2 #10). */
406 }
407 for (int i = 0; i < stages; i++) if (opened[i]) redir_finish(&redirs[i]);
408 sigchld_unblock();
409 return;
410 }
411
412 /* ----- FOREGROUND ----- */
413 take_terminal(pgid); /* E2 #2: hand the tty to the job */
414 FgResult r = wait_group(pgid, npids, 0); /* 0 = no timeout on a normal launch */
415 take_terminal(shell_pgid); /* E2 #3: reclaim the tty */
416
417 for (int i = 0; i < stages; i++) if (opened[i]) redir_finish(&redirs[i]);
418
419 if (r == FG_STOPPED) {
420     /* [E2 #5] Ctrl-Z stopped this foreground pipeline. Unlike a background
421      * launch it was never in the job table, so add it now: allocate a slot
422      * and copy the pids / pgid / names in (same fields the background branch
423      * above fills). The slot bookkeeping is below - you finish it: */
424     Job_t *slot = job_alloc();
425     if (slot != NULL) {
426         slot->pgid = pgid;
427         slot->npids = npids;
428         slot->nalive = npids;
429         memcpy(slot->pids, plist, sizeof(pid_t) * (size_t)npids);
430         for (int i = 0; i < npids; i++) strcpy(slot->pnames[i], pnm[i]);
431         str_append(slot->name, JOB_NAME_MAX, name);
432         str_append(slot->cmdline, JOB_NAME_MAX, cmdline);
433
434         /* [TODO-E2] Mark this slot's status as JS_STOPPED, then print the
435          * stopped notice in the spec's format - bracketed job id, " + ",
436          * "Stopped", then the command line, e.g. "[1] + Stopped sleep 100"
437          * (from slot->job_id and slot->cmdline). fflush() after. */
438     }
439 }
440 sigchld_unblock();
441 }
```

claude.ai

mini-project1/c-shell at main...xv6 — CS3.301C-shell project skeleton cod...https://pdos.csail.mit.edu/6.8...mit-pdos/xv6-riscv: Xv6 for...Mini Project 1 — CS3.301(5) Mod-01 Lec-17 TYPES O...

Djobs C

Copy

```
442
443 /* ===== Foreground dispatch ===== */
444 #define RUN_OK 1
445 #define RUN_FAILED 0
446
447 /* A single builtin runs IN the shell process, so redirection is done by
448  * saving fd 0/1, dup2'ing, running, then restoring. */
449 static int run_builtin_with_redir(Command *cmd)
450 {
451     Redirection redir;
452     int saved_in = -1, saved_out = -1;
453
454     if (redir_open(cmd, &redir) == 0) return RUN_OK;
455     if (redir.in_fd >= 0) { saved_in = dup(STDIN_FILENO); dup2(redir.in_fd, STDIN_FILENO); }
456     if (redir.out_fd >= 0) { fflush(stdout); saved_out = dup(STDOUT_FILENO); dup2(redir.out_fd, STDOUT_FILENO); }
457
458     builtin_run(cmd->argc, cmd->argv);
459     fflush(stdout);
460
461     if (saved_in >= 0) { dup2(saved_in, STDIN_FILENO); close(saved_in); }
462     if (saved_out >= 0) { dup2(saved_out, STDOUT_FILENO); close(saved_out); }
463     redir_finish(&redir);
464     return RUN_OK;
465 }
466
467 /* =====
468  * E1: activities - list tracked groups, oldest first, each with its members
469  *
470  * The outer selection walk below prints groups in ascending job_id (oldest
471  * first, E1 #6) and prints each group header. You fill the per-process line.
472  * =====*/
473 static void jobs_activities(void)
474 {
475     int printed;
476     int last_id = 0;
477     do {
478         int next_id = 0;
479         Job_t *pick = NULL;
480         for (int i = 0; i < JOBS_MAX; i++) { /* find the next job_id up */
481             if (jobs[i].in_use == 0) continue;
482             if (jobs[i].job_id > last_id && (next_id == 0 || jobs[i].job_id < next_id)) {
483                 next_id = jobs[i].job_id;
484                 pick = &jobs[i];
485             }
486         }
487         printed = (pick != NULL);
488         if (printed) {
489             last_id = pick->job_id;
490             printf("[%d] pgid %d\n", pick->job_id, (int)pick->pgid); /* E1 #5 group line */
491         }
492     } while (printed);
493 }
```

claude.ai

mini-project1/c-shell at main...xv6 — CS3.301C-shell project skeleton cod...https://pdos.csail.mit.edu/6.8...mit-pdos/xv6-riscv: Xv6 for...Mini Project 1 — CS3.301(5) Mod-01 Lec-17 TYPES O...

Djobs C

Copy

```
492     /* [TODO-E1] One indented line per process in this group (E1 #3/#4/#5):
493     *   for each k in [0, pick->njobs):
494     *     - skip processes that have already exited - probe liveness with
495     *       kill(pick->pids[k], 0): a non-zero return means it's gone (E1 #7).
496     *     - otherwise print, two-space indented: "  <pid> <name> <state>"
497     *       where name is pick->pnames[k] and state is "Running" or
498     *       "Stopped" from pick->status. (Column alignment doesn't matter.) */
499   }
500 } while (printed);
501 fflush(stdout);
502 }
503
504 /* =====
505 * E3: resume %n (fg [--timeout <s>] | bg) - continue a stopped/bg job
506 *
507 * All the argument validation and the fixed error strings (E3 #8/#9) are done.
508 * You implement the job-control actions (E3 #2/#3/#4/#5/#6/#10/#11).
509 * =====*/
510 static void jobs_resume(int argc, char **argv)
511 {
512     /* --- syntax (E3 #9): resume %<digits> fg|bg [--timeout <digits>] --- */
513     if (argc < 3 || argv[1][0] != '%' || all_digits(argv[1] + 1) == 0) {
514         fprintf(stderr, "resume: invalid syntax\n"); return;
515     }
516     int is_fg;
517     if (strcmp(argv[2], "fg") == 0) is_fg = 1;
518     else if (strcmp(argv[2], "bg") == 0) is_fg = 0;
519     else { fprintf(stderr, "resume: invalid syntax\n"); return; }
520
521     int timeout = 0;
522     if (is_fg && argc > 3) {
523         if (argc != 5 || strcmp(argv[3], "--timeout") != 0 || all_digits(argv[4]) == 0) {
524             fprintf(stderr, "resume: invalid syntax\n"); return;
525         }
526         timeout = atoi(argv[4]);
527     } else if (!is_fg && argc != 3) {
528         fprintf(stderr, "resume: invalid syntax\n"); return;
529     }
530
531     Job_t *j = job_find_by_id(atoi(argv[1] + 1));
532     if (j == NULL) { fprintf(stderr, "resume: no such job\n"); return; } /* E3 #8 */
533
534     /* [TODO-E3] From here the actions are yours:
535     * 1. Wake the whole group: send SIGCONT to -j->pgid, and set j->status
536     *    = JS_RUNNING (E3 #2).
537     * 2. bg branch (E3 #4/#10): print "[%d] + Running  %s" with j->job_id
538     *    and j->cmdline, fflush, and return immediately - do NOT touch the
539     *    terminal or wait.
540     * 3. fg branch (E3 #3/#5/#6/#7/#11):
541     *    - First print the job's command line on its own (E3 #11), like a
```

claude.ai

mini-project1/c-shell at main...xv6 — CS3.301C-shell project skeleton cod...https://pdos.csail.mit.edu/6.8...mit-pdos/xv6-riscv: Xv6 for...Mini Project 1 — CS3.301(5) Mod-01 Lec-17 TYPES O...

Djobs C

Copy

```
543 * - Snapshot what you need (pgid, naive, job_id, cmdline) into
544 * locals, because the job slot may be freed while you wait.
545 * - sigchld_block(); take_terminal(pgid);
546 * FgResult r = wait_group(pgid, naive, timeout); // timeout arms alarm
547 * take_terminal(shell_pgid);
548 * - Re-find the job by id (it may have vanished), then on:
549 * FG_DONE -> free the slot (in_use = 0).
550 * FG_STOPPED -> set status JS_STOPPED and print "[%d] + Stopped %s".
551 * FG_TIMEOUT -> free the slot and print "resume: job timed out" (E3 #6).
552 * fflush; sigchld_unblock();
553 * (timeout is 0 for bg and for fg without --timeout, so the alarm path
554 * only engages when the user asked for it.) */
555 (void)timeout; (void)is_fg; /* remove once used in TODO-E3 */
556 }
557
558 /* =====
559 * E4: ping <target> <signal_number> - send a signal to a pid or a %job
560 *
561 * Argument-count and the "validate the signal before the target" ordering
562 * (E4 #3) plus the modulo (E4 #2) are set up. You resolve the target, check it
563 * is one this shell is tracking, deliver the signal, and print the result.
564 * =====*/
565 static void jobs_ping(int argc, char **argv)
566 {
567     if (argc != 3) { fprintf(stderr, "ping: invalid syntax\n"); return; }
568
569     /* E4 #3/#7: the signal is validated FIRST. A non-negative integer is the
570     * only valid form; a leading '-' fails all_digits, so negatives correctly
571     * fall out here as "invalid syntax" rather than being reduced by modulo. */
572     if (all_digits(argv[2]) == 0) { fprintf(stderr, "ping: invalid syntax\n"); return; }
573     int sig = atoi(argv[2]); /* the number the user typed (echo this, E4 #6) */
574     int actual = sig % 64; /* the signal actually delivered (E4 #2) */
575
576     const char *tgt = argv[1];
577
578     /* [TODO-E4] Resolve the target, deliver 'actual', and report:
579     * - If tgt starts with '%': the rest must be digits (else "ping: no such
580     * process found"); look the job up with job_find_by_id and, if found,
581     * send 'actual' to the whole group with kill(-pgid, actual).
582     * - Otherwise tgt must be digits forming a pid; only accept it if THIS
583     * shell is tracking it - job_find_by_pid(pid) must be non-NULL (E4 #8) -
584     * then kill(pid, actual).
585     * - Any unknown target, or a kill() that fails, prints exactly:
586     * "ping: no such process found"
587     * - On success print: "Sent signal %d to %s" (or "... to %d" for a pid),
588     * echoing 'sig' - the ORIGINAL number, never 'actual' (E4 #5/#6).
589     * - fflush at the end. */
590     (void)tgt; (void)actual; /* remove once used in TODO-E4 */
591 }
592
```

claude.ai

mini-project1/c-shell at main...xv6 — CS3.301C-shell project skeleton cod...https://pdos.csail.mit.edu/6.8...mit-pdos/xv6-riscv: Xv6 for...Mini Project 1 — CS3.301(5) Mod-01 Lec-17 TYPES O...

Djobs CCopy

```
593 static int run_foreground(Job *job)
594 {
595     if (job->command_count == 1) {
596         Command *cmd = job->commands[0];
597         char path[EXEC_PATH_MAX];
598
599         if (cmd->argc == 0) return RUN_OK;
600
601         /* job-control + custom builtins run in the shell (they need the table) */
602         if (strcmp(cmd->argv[0], "activities") == 0) { jobs_activities(); return RUN_OK; }
603         if (strcmp(cmd->argv[0], "resume") == 0) { jobs_resume(cmd->argc, cmd->argv); return RUN_OK; }
604         if (strcmp(cmd->argv[0], "ping") == 0) { jobs_ping(cmd->argc, cmd->argv); return RUN_OK; }
605         if (strcmp(cmd->argv[0], "spy") == 0) { spy_run(cmd->argc, cmd->argv); return RUN_OK; }
606         if (strcmp(cmd->argv[0], "snoop") == 0) { snoop_run(cmd->argc, cmd->argv); return RUN_OK; }
607
608         if (builtin_is_builtin(cmd->argv[0]) == 1)
609             return run_builtin_with_redir(cmd);
610
611         /* [TODO-5] D1 rule 3: a single external command that CANNOT be started
612          * at all must NOT launch, and must STOP the ';' sequence. Try to
613          * resolve it into 'path' with exec_resolve() first; if that fails,
614          * print the standard "cshell: command not found (name)" line
615          * (SHELL_NAME + bare_name()) and return RUN_FAILED - that return is
616          * what makes jobs_run_sequence() break out of the sequence.
617          * Note the boundary: a command that DOES start and then exits with a
618          * non-zero status has NOT failed (D1 rule 4) - the sequence keeps
619          * going - so only the "can't resolve" case returns RUN_FAILED here. */
620         (void)path; /* remove once you use it in TODO-5 */
621     }
622     launch_pipeline(job, 0);
623     return RUN_OK;
624 }
625
626 /* ===== Public API ===== */
627 static void install(int signo, void (*handler)(int), int restart)
628 {
629     struct sigaction sa;
630     memset(&sa, 0, sizeof(sa));
631     sa.sa_handler = handler;
632     sigemptyset(&sa.sa_mask);
633     sa.sa_flags = restart ? SA_RESTART : 0;
634     sigaction(signo, &sa, NULL);
635 }
636
637 /* Call once at startup (already wired into a_main.c). Installs the SIGCHLD
638  * reaper (D2 #6) and puts the shell in charge of the terminal. */
639 void jobs_init(void)
640 {
641     for (int i = 0; i < JOBS_MAX; i++) jobs[i].in_use = 0;
642 }
```

claude.ai

mini-project1/c-shell at main...xv6 — CS3.301C-shell project skeleton cod...https://pdos.csail.mit.edu/6.8...mit-pdos/xv6-riscv: Xv6 for...Mini Project 1 — CS3.301(5) Mod-01 Lec-17 TYPES O...

Djobs C

Copy

```
643     install(SIGCHLD, on_sigchld, 1); /* SA_RESTART so read() isn't broken */
644     install(SIGINT, on_sigint, 0); /* no restart => read() gets EINTR */
645     install(SIGTSTP, on_sigstsp, 1); /* E2 #1: catch+ignore, shell won't stop */
646     install(SIGALRM, on_sigalrm, 0); /* E3: no restart so wait_group sees EINTR */
647     signal(SIGTTOU, SIG_IGN); /* E2 #1: tcsetpgrp won't stop the shell */
648     signal(SIGTTIN, SIG_IGN);
649
650     have_tty = isatty(STDIN_FILENO);
651     setpgid(0, 0);
652     shell_pgid = getpgrp();
653     if (have_tty) tcsetpgrp(STDIN_FILENO, shell_pgid);
654 }
655
656 /* D1: run every job in the list left-to-right. Backgrounds are launched and
657  * not waited for; foregrounds are waited for; a foreground that cannot START
658  * stops the rest of the sequence (D1 rule 3). */
659 void jobs_run_sequence(Joblist *list, const char *raw_line)
660 {
661     (void)raw_line; /* Part E uses this to store cmdline */
662     for (int i = 0; i < list->jobs_count; i++) {
663         Job *job = list->jobs[i];
664         if (job->command_count == 0) continue;
665         if (job->command_count == 1 && job->commands[0]->argc == 0) continue;
666
667         if (job->background == 1) {
668             launch_pipeline(job, 1);
669         } else {
670             if (run_foreground(job) == RUN_FAILED)
671                 break; /* D1 rule 3 */
672         }
673     }
674 }
675
676 int jobs_has_stopped(void)
677 {
678     for (int i = 0; i < JOBS_MAX; i++)
679         if (jobs[i].in_use == 1 && jobs[i].status == JS_STOPPED)
680             return 1;
681     return 0;
682 }
683
684 void jobs_hangup_all(void)
685 {
686     for (int i = 0; i < JOBS_MAX; i++) {
687         if (jobs[i].in_use == 0) continue;
688         kill(-jobs[i].pgid, SIGHUP);
689         kill(-jobs[i].pgid, SIGCONT); /* so a stopped group receives it */
690     }
691 }
692
```

🔍 mini-project1/c-shell at main...

💰 xv6 — CS3.301

🔥 C-shell project skeleton cod...

🌐 https://pdos.csail.mit.edu/6.8...

🔍 mit-pdos/xv6-riscv: Xv6 for...

📄 Mini Project 1 — CS3.301

📺 (5) Mod-01 Lec-17 TYPES O...

📄 C-shell project skeleton code ⌵

📄 Share

Give the heavy skeleton for part F and tell how to do the rest

Ran a command >

Part F is two files — `f_spy.c` (F1: read `/proc`) and `f_snoop.c` (F2: `ptrace` tracer). Let me build both skeletons.

Created 2 files, ran a command, shared files >

How to complete each `TODO`:

F1 — `emit_fds()` in `f_spy.c` (one function, two gaps). `/proc/<pid>/fd` holds one entry per open descriptor; the entry name is the fd number and it's a symlink to whatever that fd points at. (a) Walk the directory with `readdir()`, keeping only entries whose name is numeric (that skips `.` and `..`), and `atoi()` each into `fds[n++]`. (b) After the provided sort, for each fd build `/proc/<pid>/fd/<n>`, `read_link()` it, and on success emit a row where the label is the fd number as text, the type comes from `type_of()` on the link path (stat follows it to the real object), and the path is the link target. `emit_link()` and `emit_maps()` right above it are the same pattern worked out in full — copy their shape. Test: `spy` with no argument lists the shell's own cwd, exe, mapped libraries, and fds (0/1/2 → your terminal); `spy <pid>` of a running process matches; a bad pid → `spy: no such process`.

F2 — the `ptrace` calls in `f_snoop.c` (four small gaps). The stepping loop, syscall table, timing, aggregation, sort, and print are all done; you supply the four `ptrace` calls that make tracing actually happen:

- **F2b (child, command mode):** before `execv`, the child calls `ptrace(PTRACE_TRACEME, 0, 0, 0)`. That's what makes it stop on its `execve` and hand control to the parent — without it the parent's `waitpid` has nothing to catch.
- **F2d (attach mode):** `ptrace(PTRACE_ATTACH, pid, 0, 0)`; if it fails, print `snoop: no such process` and return 1. Attaching injects a stop that the following `waitpid` collects.
- **F2c (both modes):** after that first stop, `ptrace(PTRACE_SETOPTIONS, pid, 0, PTRACE_O_TRACESYSGOOD)`. This is what makes syscall stops arrive as `SIGTRAP | 0x80`, which is exactly the bit `run_loop` tests to tell a syscall-stop from an ordinary signal.
- **F2a (inside `run_loop`, syscall entry):** declare a `struct user_regs_struct regs`, call `ptrace(PTRACE_GETREGS, pid, 0, ®s)`, and set `cur = (long)regs.orig_rax`. Use `orig_rax`, not `rax` — the kernel overwrites `rax` with the return value, but `orig_rax` preserves the syscall number across the call.

Test: `snoop ls` prints a table of syscalls with counts and time (expect `execve`, `openat`, `read`, `write`, `close`, `mmap`, `exit_group`); `snoop -p ↓ d>` of a shell-spawned process attaches and traces until it exits; a bad command → `snoop: command not found`; a bad pid → `snoop:`

+ Write a message...

🔊

🔄

⌵

claude.ai

mini-project1/c-shell at main...xv6 — CS3.301C-shell project skeleton cod...https://pdos.csail.mit.edu/6.8...mit-pdos/xv6-riscv: Xv6 for...Mini Project 1 — CS3.301(5) Mod-01 Lec-17 TYPES O...

F spy C

Copy

```
1  /*
2   * f_spy.c - F1: spy [pid]
3   *
4   * Lists the open files of a process (or the shell itself, when no pid is given)
5   * by reading /proc/<pid>/. Prints a table: PID, FD, TYPE, PATH - covering the
6   * cwd, the executable (txt), memory-mapped files, and the numbered descriptors.
7   *
8   * Implemented here: the stat->TYPE mapping, the readlink/print helpers, the cwd
9   * and exe symlink rows, the /proc/<pid>/maps reader, and spy_run's argument
10  * parsing / existence + permission checks / table header.
11  *
12  * Marked TODO: emit_fds() - reading /proc/<pid>/fd. Use emit_link() and
13  * emit_maps() below as worked examples of the same /proc pattern.
14  *
15  * Compiles as-is under -std=c23 -Wall -Wextra -Werror.
16  */
17
18 #include "f_spy.h"
19 #include "a_shell.h"
20
21 #include <dirent.h>
22 #include <stdio.h>
23 #include <stdlib.h>
24 #include <string.h>
25 #include <sys/stat.h>
26 #include <unistd.h>
27 #include <errno.h>
28
29 /* Map a path's stat mode to the short TYPE label spy prints. */
30 static const char *type_of(const char *path)
31 {
32     struct stat s;
33     if (stat(path, &s) != 0) return "unknown";
34     if (S_ISREG(s.st_mode)) return "REG";
35     if (S_ISDIR(s.st_mode)) return "DIR";
36     if (S_ISCHR(s.st_mode)) return "CHR";
37     if (S_ISBLK(s.st_mode)) return "BLK";
38     if (S_ISFIFO(s.st_mode)) return "FIFO";
39     if (S_ISSOCK(s.st_mode)) return "unix";
40     if (S_ISLNK(s.st_mode)) return "LINK";
41     return "unknown";
42 }
43
44 /* readlink() wrapper that NUL-terminates. Returns 1 on success, 0 on failure. */
45 static int read_link(const char *p, char *buf, size_t n)
46 {
47     ssize_t r = readlink(p, buf, n - 1);
48     if (r < 0) return 0;
49     buf[r] = '\0';
50     return 1;
51 }
```



claude.ai

mini-project1/c-shell at main...

xv6 — CS3.301

C-shell project skeleton cod...

https://pdos.csail.mit.edu/6.8...

mit-pdos/xv6-riscv: Xv6 for...

Mini Project 1 — CS3.301

(5) Mod-01 Lec-17 TYPES O...

F spy C

Copy

```
52
53 /* One table row. */
54 static void row(pid_t pid, const char *fd, const char *type, const char *path)
55 {
56     printf("%-6d %-6s %-6s %s\n", (int)pid, fd, type, path);
57 }
58
59 /* A single symlink under /proc/<pid>/ (used for cwd and exe). Worked example of
60  * the pattern emit_fds() needs: build the /proc path, readlink it, print a row. */
61 static void emit_link(pid_t pid, const char *label, const char *rel)
62 {
63     char link[128], tgt[4096];
64     snprintf(link, sizeof link, "/proc/%d/%s", (int)pid, rel);
65     if (read_link(link, tgt, sizeof tgt))
66         row(pid, label, type_of(tgt), tgt);
67 }
68
69 /* Each unique memory-mapped file path from /proc/<pid>/maps, printed once.
70  * Worked example of reading and parsing a /proc text file. */
71 static void emit_maps(pid_t pid)
72 {
73     char path[128];
74     snprintf(path, sizeof path, "/proc/%d/maps", (int)pid);
75     FILE *f = fopen(path, "r");
76     if (f == NULL) return;
77
78     char seen[512][4096];
79     int nseen = 0;
80     char line[8192];
81
82     while (fgets(line, sizeof line, f) != NULL) {
83         char *p = strchr(line, '/'); /* pathname starts at first '/' */
84         if (p == NULL) continue; /* anon / [heap] / [stack] */
85         size_t len = strlen(p);
86         if (len > 0 && p[len - 1] == '\n') p[--len] = '\0';
87
88         int dup = 0;
89         for (int i = 0; i < nseen; i++)
90             if (strcmp(seen[i], p) == 0) { dup = 1; break; }
91         if (dup) continue;
92
93         if (nseen < 512) {
94             strncpy(seen[nseen], p, 4095);
95             seen[nseen][4095] = '\0';
96             nseen++;
97         }
98         row(pid, "mem", type_of(p), p);
99     }
100     fclose(f);
101 }
```

claude.ai

mini-project1/c-shell at main...xv6 — CS3.301C-shell project skeleton cod...https://pdos.csail.mit.edu/6.8...mit-pdos/xv6-riscv: Xv6 for...Mini Project 1 — CS3.301(5) Mod-01 Lec-17 TYPES O...

F spy C

Copy

```
102
103 static int cmp_int(const void *a, const void *b)
104 {
105     int x = *(const int *)a, y = *(const int *)b;
106     return (x > y) - (x < y);
107 }
108
109 /* =====
110 * [TODO-F1] emit_fds() - the numbered open descriptors, ascending
111 *
112 * The directory /proc/<pid>/fd contains one entry per open descriptor; the
113 * entry name is the fd number, and it is a symlink to whatever that fd refers
114 * to (a file, pipe, socket, tty, ...).
115 *
116 * Two gaps to fill (the opendir/closedir + qsort scaffolding is provided):
117 * (a) Read the directory with readdir(). Keep only entries whose name is a
118 *     number (skip "." / ".."), converting each with atoi() into fds[n++].
119 * (b) After the sort, for each fd: build "/proc/<pid>/fd/<n>", read_link()
120 *     it into a buffer, and if that succeeds emit a row: the label is the fd
121 *     number as text, the type comes from type_of() on the LINK path itself
122 *     (stat follows it to the real object), and the path is the link target.
123 *
124 * Model both gaps on emit_link()/emit_maps() above - same helpers, same shape.
125 * =====*/
126 static void emit_fds(pid_t pid)
127 {
128     char dirp[128];
129     snprintf(dirp, sizeof dirp, "/proc/%d/fd", (int)pid);
130     DIR *d = opendir(dirp);
131     if (d == NULL) return;
132
133     int fds[1024];
134     int n = 0;
135
136     /* TODO-F1(a): readdir(d) and collect numeric entry names into fds[0..n). */
137
138     closedir(d);
139     qsort(fds, n, sizeof(int), cmp_int);
140
141     /* TODO-F1(b): for each fds[i], read the /proc/<pid>/fd/<i> symlink and,
142      * on success, row(pid, "<fd number>", type_of(link), target). */
143 }
144
145 int spy_run(int argc, char **argv)
146 {
147     pid_t pid;
148
149     if (argc == 1) pid = getpid(); /* no arg -> the shell itself */
150     else if (argc == 2) pid = (pid_t)atoi(argv[1]);
151     else { fprintf(stderr, "spy: invalid syntax\n"); return 1; }
```



claude.ai

mini-project1/c-shell at main...xv6 — CS3.301C-shell project skeleton cod...https://pdos.csail.mit.edu/6.8...mit-pdos/xv6-riscv: Xv6 for...Mini Project 1 — CS3.301(5) Mod-01 Lec-17 TYPES O...

F spy C

Copy

```
123 *
124 * Model both gaps on emit_link()/emit_maps() above — same helpers, same shape.
125 * =====*/
126 static void emit_fds(pid_t pid)
127 {
128     char dirp[128];
129     snprintf(dirp, sizeof dirp, "/proc/%d/fd", (int)pid);
130     DIR *d = opendir(dirp);
131     if (d == NULL) return;
132
133     int fds[1024];
134     int n = 0;
135
136     /* TODO-F1(a): readdir(d) and collect numeric entry names into fds[0..n). */
137
138     closedir(d);
139     qsort(fds, n, sizeof(int), cmp_int);
140
141     /* TODO-F1(b): for each fds[i], read the /proc/<pid>/fd/<i> symlink and,
142      *      on success, row(pid, "<fd number>", type_of(link), target). */
143 }
144
145 int spy_run(int argc, char **argv)
146 {
147     pid_t pid;
148
149     if (argc == 1) pid = getpid();          /* no arg -> the shell itself */
150     else if (argc == 2) pid = (pid_t)atoi(argv[1]);
151     else { fprintf(stderr, "spy: invalid syntax\n"); return 1; }
152
153     char base[64];
154     struct stat st;
155     snprintf(base, sizeof base, "/proc/%d", (int)pid);
156     if (stat(base, &st) != 0) { fprintf(stderr, "spy: no such process\n"); return 1; }
157
158     char fddir[80];
159     snprintf(fddir, sizeof fddir, "/proc/%d/fd", (int)pid);
160     DIR *probe = opendir(fddir);
161     if (probe == NULL && errno == EACCES) { fprintf(stderr, "spy: permission denied\n"); return 1; }
162     if (probe != NULL) closedir(probe);
163
164     printf("%-6s %-6s %-6s %s\n", "PID", "FD", "TYPE", "PATH");
165     emit_link(pid, "cwd", "cwd");
166     emit_link(pid, "txt", "exe");
167     emit_maps(pid);
168     emit_fds(pid);
169     fflush(stdout);
170     return 0;
171 }
172
```

claude.ai

mini-project1/c-shell at main...

xv6 — CS3.301

C-shell project skeleton cod...

https://pdos.csail.mit.edu/6.8...

mit-pdos/xv6-riscv: Xv6 for...

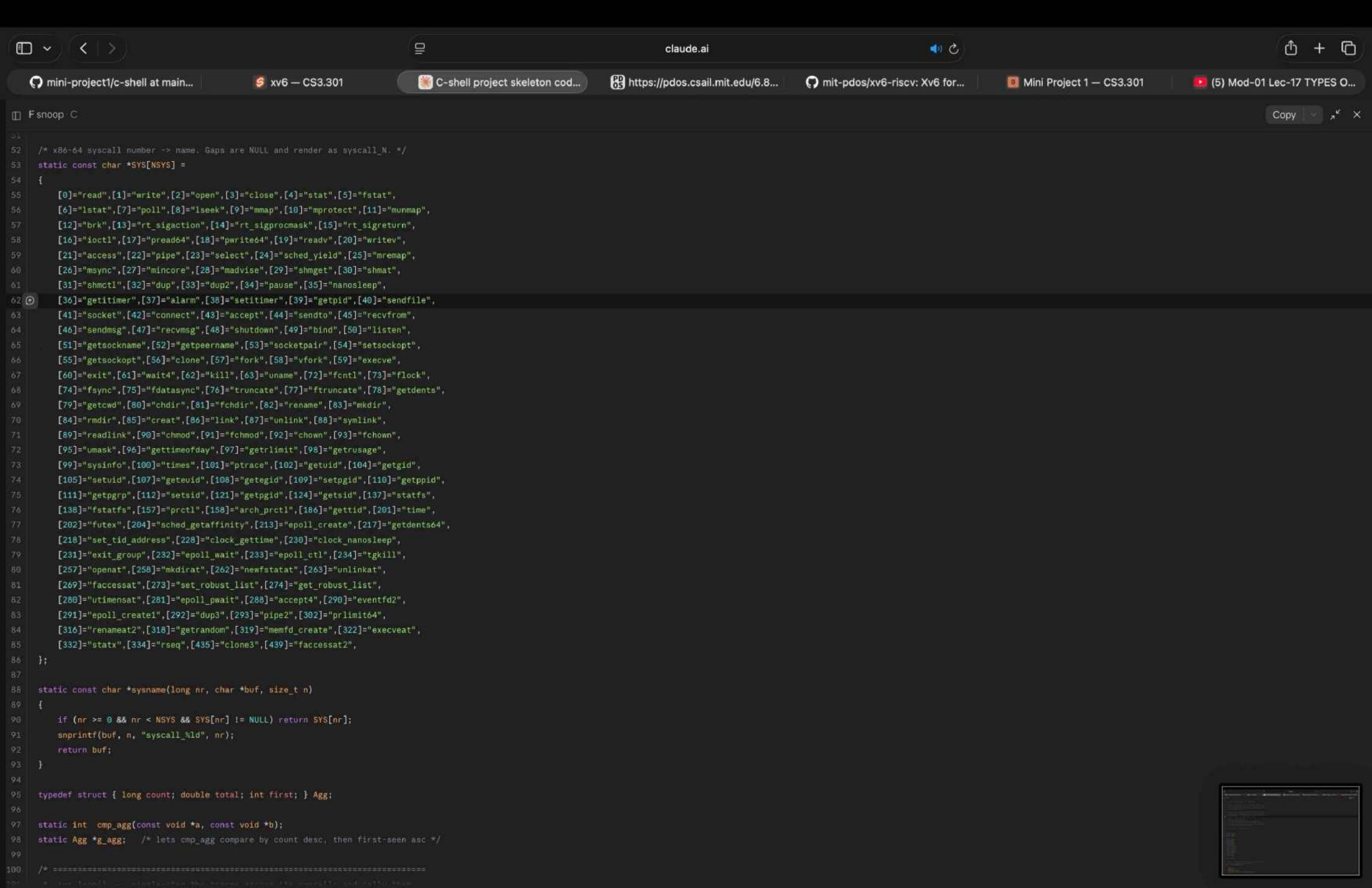
Mini Project 1 — CS3.301

(5) Mod-01 Lec-17 TYPES O...

F snoop.c

Copy

```
1  /*
2   * f_snoop.c - F2: snoop <cmd [args...]> | snoop -p <pid>
3   *
4   * Traces a process with ptrace, counts every system call it makes (with total
5   * time spent in each), and prints a summary when the tracee exits. Two modes:
6   * launch a command and trace it from the first execve, or attach to a running
7   * pid the shell already knows about.
8   *
9   * Implemented here: the SIGCHLD guard (so the shell's own reaper doesn't steal
10  * the tracee's stops), the x86-64 syscall-number->name table and lookup, the
11  * per-syscall aggregation + count-desc/first-seen sort, the summary print, and
12  * snoop_run's argument parsing / command resolution / fork.
13  *
14  * Marked TODO - the ptrace mechanics:
15  *   F2a : read the syscall number on each entry stop (PTRACE_GETREGS).
16  *   F2b : the traced child announces itself (PTRACE_TRACEME) before execve.
17  *   F2c : after the first stop, ask for syscall-good stops (PTRACE_SETOPTIONS).
18  *   F2d : attach mode grabs the target (PTRACE_ATTACH).
19  *
20  * Compiles as-is under -std=c23 -Wall -Wextra -Werror.
21  */
22
23  #include "f_snoop.h"
24  #include "a_shell.h"
25  #include "c_exec.h"
26
27  #include <errno.h>
28  #include <signal.h>
29  #include <stdio.h>
30  #include <stdlib.h>
31  #include <string.h>
32  #include <sys/ptrace.h>
33  #include <sys/stat.h>
34  #include <sys/types.h>
35  #include <sys/user.h>
36  #include <sys/wait.h>
37  #include <time.h>
38  #include <unistd.h>
39
40  #define NSYS 512
41
42  /* The shell's SIGCHLD handler would reap our tracee's stops; block SIGCHLD for
43   * the duration of tracing so ptrace sees every wait status. */
44  static void snoop_sigchld(int block)
45  {
46      sigset_t s;
47      sigemptyset(&s);
48      sigaddset(&s, SIGCHLD);
49      sigprocmask(block ? SIG_BLOCK : SIG_UNBLOCK, &s, NULL);
50  }
```



claude.ai

mini-project1/c-shell at main...

xv6 — CS3.301

C-shell project skeleton cod...

https://pdos.csail.mit.edu/6.8...

mit-pdos/xv6-riscv: Xv6 for...

Mini Project 1 — CS3.301

(5) Mod-01 Lec-17 TYPES O...

F snoop · C

Copy

```
99
100 /* =====
101  * run_loop() - single-step the traces across its syscalls and tally them
102  *
103  * The stepping mechanics are provided: PTRACE_SYSCALL runs the traces to the
104  * next syscall-stop, waitpid() collects it, and WSTOPSIG == (SIGTRAP|0x80)
105  * identifies a syscall-good stop (that bit is why F2c sets TRACESYSGOOD). Each
106  * syscall produces TWO such stops - entry then exit - which 'in_call' toggles
107  * between. The aggregation, timing, sort, and print are all provided.
108  *
109  * [TODO-F2a] is the one gap: on a syscall ENTRY stop, read which syscall it is.
110  * =====*/
111 static void run_loop(pid_t pid)
112 {
113     Agg agg[NSYS];
114     long order_nr[NSYS];
115     int norder = 0;
116     memset(agg, 0, sizeof agg);
117
118     int in_call = 0;
119     long cur = -1;
120     struct timespec t0 = {0, 0};
121     int status;
122
123     while (1) {
124         if (ptrace(PTRACE_SYSCALL, pid, 0, 0) < 0) break; /* run to next syscall-stop */
125         if (waitpid(pid, &status, 0) < 0) break;
126         if (WIFEXITED(status) || WIFSIGNALED(status)) break;
127
128         if (WIFSTOPPED(status) && WSTOPSIG(status) == (SIGTRAP | 0x80)) {
129             if (in_call == 0) { /* ---- syscall entry ---- */
130                 /* TODO-F2a: read the syscall number into 'cur'. Declare a
131                  * 'struct user_regs_struct regs;', call
132                  * ptrace(PTRACE_GETREGS, pid, 0, &regs), and on success take
133                  * 'cur = (long)regs.orig_rax;' (orig_rax holds the syscall
134                  * number even after the kernel overwrites rax with the result). */
135                 cur = -1; /* placeholder until TODO-F2a */
136                 clock_gettime(CLOCK_MONOTONIC, &t0);
137                 in_call = 1;
138             } else { /* ---- syscall exit ---- */
139                 struct timespec t1;
140                 clock_gettime(CLOCK_MONOTONIC, &t1);
141                 double d = (double)(t1.tv_sec - t0.tv_sec)
142                     + (double)(t1.tv_nsec - t0.tv_nsec) / 1e9;
143                 if (cur >= 0 && cur < NSYS) {
144                     if (agg[cur].count == 0) { agg[cur].first = norder; order_nr[norder++] = cur; }
145                     agg[cur].count++;
146                     agg[cur].total += d;
147                 }
148                 in_call = 0;
149             }
150         }
151     }
152 }
```

Copy

F snoop C

claude.ai

mini-project1/c-shell at main...xv6 — CS3.301C-shell project skeleton cod...https://pdos.csail.mit.edu/6.8...mit-pdos/xv6-riscv: Xv6 for...Mini Project 1 — CS3.301(5) Mod-01 Lec-17 TYPES O...

F snoop.cCopy

```
182     struct stat st;
183     snprintf(proc, sizeof proc, "/proc/%d", (int)pid);
184     if (stat(proc, &st) != 0) { fprintf(stderr, "snoop: no such process\n"); return 1; }
185
186     /* TODO-F2d: attach to the target with ptrace(PTRACE_ATTACH, pid, 0, 0).
187      * If it fails, print "snoop: no such process" and return 1. Attaching
188      * sends the tracee a stop that the waitpid() below then collects. */
189
190     int status;
191     snoop_sigchld(1);
192     waitpid(pid, &status, 0); /* the attach stop */
193     /* TODO-F2c: ptrace(PTRACE_SETOPTIONS, pid, 0, PTRACE_O_TRACESYSGOOD)
194      * so syscall stops arrive as (SIGTRAP|0x80), which run_loop checks. */
195     run_loop(pid);
196     snoop_sigchld(0);
197     return 0;
198 }
199
200 /* ---- command mode: snoop <cmd> [args...] ---- */
201 if (argc >= 2) {
202     char path[EXEC_PATH_MAX];
203     if (exec_resolve(argv[1], path, EXEC_PATH_MAX) == 0) {
204         fprintf(stderr, "snoop: command not found\n");
205         return 1;
206     }
207     pid_t pid = fork();
208     if (pid < 0) { perror("snoop: fork"); return 1; }
209
210     if (pid == 0) { /* ---- tracee ---- */
211         snoop_sigchld(0); /* clean mask for the child */
212         /* TODO-F2b: ptrace(PTRACE_TRACEME, 0, 0, 0) so this child becomes
213          * traceable; it will then stop on the execve below and hand
214          * control back to the parent. */
215         execv(path, &argv[1]);
216         _exit(127);
217     }
218
219     int status;
220     snoop_sigchld(1);
221     waitpid(pid, &status, 0); /* stop after execve */
222     /* TODO-F2c: PTRACE_SETOPTIONS with PTRACE_O_TRACESYSGOOD (as above). */
223     run_loop(pid);
224     snoop_sigchld(0);
225     return 0;
226 }
227
228 fprintf(stderr, "snoop: invalid syntax\n");
229 return 1;
230 }
231
```


Implementing MLFQ scheduler in xv6

Ran 9 commands

I now have a complete picture of the existing machinery. Let me note the key facts I've confirmed, because the design depends on them:

- Timer ticks call `yield()` **unconditionally** every tick in both `usertrap` and `kerneltrap` (`trap.c`). This is RR. MLFQ must replace this with tick-accounting logic.
- `allocproc` is the single choke point for process creation; `kfork` sets the new proc `RUNNABLE` at the end.
- Voluntary yield happens via `sleep()` (RUNNING→SLEEPING); becoming runnable again happens in `wakeup()` (SLEEPING→RUNNABLE). These are the hooks for Rule 5.
- `kexit` → ZOMBIE. The scheduler only ever runs `RUNNABLE` procs.
- Global `ticks` lives in `trap.c`; `clockintr` bumps it on `cpuid 0` — the natural place for the 48-tick boost.

Let me check the Makefile's CFLAGS and CPU count so I know how to wire the `SCHEDULER` macro.

Ran a command

Good — CPUS defaults to 3, but MLFQ's strict global priority is cleanest and standard to evaluate with `CPUS=1`, which I'll use for testing.

I now have everything I need. Let me lay out the plan before touching code so you can follow the theory:

MLFQ design (why each piece):

1. **Represent the 4 queues** by a `priority` field (0-3) on each process, with FIFO order *within* a queue tracked by a monotonic `enter_seq` counter. This is provably equivalent to explicit array-queues but avoids xv6's nastiest deadlock traps (holding `p->lock` while touching a shared queue in `sleep / wakeup`). I'll document this choice for the viva.
2. **Selection** (Rule 2, strict priority + Rule 6, RR in Q3): scheduler scans for the RUNNABLE proc with the lowest `priority`, breaking ties by lowest `enter_seq` (= oldest in queue = FIFO/round-robin).
3. **Time slices** `{1,4,8,16}` counted per tick (Rule 2.1.2).
4. **Slice exhaustion** (Rule 4): on tick, if `ticks_used` hits the slice, demote to next lower queue (or stay at Q3), re-inserted at tail (new `enter_seq`).
5. **Preemption** (Rule 2): on tick, if a higher-priority queue is non-empty, yield without demoting.
6. **Voluntary yield** (Rule 5): `sleep()` resets `ticks_used` (original gameable MLFQ rule — key for the report's I/O-bound analysis); `wakeup()` re-inserts at tail of the *same* queue, priority unchanged.
7. **Boost** (Rule 7): every 48 ticks, in `clockintr`, " processes → Q0.
8. `#ifdef MLFQ` guards everywhere so the RR fallback compiles unchanged when `SCHEDULER` is

Mlfq trace · LOG

Copy

```

1  [MLFQ] t=0 pid=1 q=0 ev=NEW
2  [MLFQ] t=1 pid=1 q=0 ev=SLEEP
3  [MLFQ] t=1 pid=1 q=0 ev=WAKE
4  [MLFQ] t=1 pid=1 q=0 ev=SLEEP
5  [MLFQ] t=1 pid=1 q=0 ev=WAKE
6  [MLFQ] t=1 pid=1 q=0 ev=SLEEP
7  [MLFQ] t=1 pid=1 q=0 ev=WAKE
8  [MLFQ] t=1 pid=1 q=0 ev=SLEEP
9  [MLFQ] t=1 pid=1 q=0 ev=WAKE
10 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
11 [MLFQ] t=1 pid=1 q=0 ev=WAKE
12 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
13 [MLFQ] t=1 pid=1 q=0 ev=WAKE
14 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
15 [MLFQ] t=1 pid=1 q=0 ev=WAKE
16 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
17 [MLFQ] t=1 pid=1 q=0 ev=WAKE
18 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
19 [MLFQ] t=1 pid=1 q=0 ev=WAKE
20 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
21 [MLFQ] t=1 pid=1 q=0 ev=WAKE
22 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
23 [MLFQ] t=1 pid=1 q=0 ev=WAKE
24 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
25 [MLFQ] t=1 pid=1 q=0 ev=WAKE
26 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
27 [MLFQ] t=1 pid=1 q=0 ev=WAKE
28 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
29 [MLFQ] t=1 pid=1 q=0 ev=WAKE
30 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
31 [MLFQ] t=1 pid=1 q=0 ev=WAKE
32 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
33 [MLFQ] t=1 pid=1 q=0 ev=WAKE
34 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
35 [MLFQ] t=1 pid=1 q=0 ev=WAKE
36 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
37 [MLFQ] t=1 pid=1 q=0 ev=WAKE
38 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
39 [MLFQ] t=1 pid=1 q=0 ev=WAKE
40 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
41 [MLFQ] t=1 pid=1 q=0 ev=WAKE
42 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
43 [MLFQ] t=1 pid=1 q=0 ev=WAKE
44 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
45 [MLFQ] t=1 pid=1 q=0 ev=WAKE
46 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
47 [MLFQ] t=1 pid=1 q=0 ev=WAKE
48 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
49 [MLFQ] t=1 pid=1 q=0 ev=WAKE
50 [MLFQ] t=1 pid=1 q=0 ev=SLEEP

```

- Implementing MLFQ scheduler in xv6

6. **Voluntary yield** (Rule 5): `sleep()` resets `ticks_used` (original gameable MLFQ rule — key for the report's I/O-bound analysis); `wakeup()` re-inserts at tail of the *same* queue, priority unchanged.
7. **Boost** (Rule 7): every 48 ticks, in `clockintr`, all processes $\rightarrow$ Q0.
8. **#ifdef MLFQ guards** everywhere so the RR path compiles unchanged when `SCHEDULER` is unset.

Let me verify the include chains before editing (so my new symbols resolve).

Edited 6 files, ran 14 commands >

Status: core MLFQ implemented and verified

I have the RISC-V toolchain + QEMU installed and a working **build → boot → run → verify** loop. The core MLFQ scheduler (Section 2.1, the [70]-mark bulk) is written and passes every check I can run so far.

What's done and verified:

- Both configs build clean with `-Wall -Werror: make clean; make qemu` (RR, unchanged path) and `make clean; make qemu SCHEDULER=MLFQ`. Confirmed `-D MLFQ` actually reaches the compiler.
- MLFQ kernel boots and runs userspace (`echo`, `ls`, fork/exec, console + disk I/O) with no panic on `CPU=1`.
- `procdump` (`Ctrl+P`) works and prints the new columns: PID, state, name, queue, ticks used in slice, enqueue-seq, creation tick.

What each change implements (all `#ifdef MLFQ`-guarded so RR is untouched):

- **Makefile** — `SCHEDULER=MLFQ` macro; `unset → RR`.
- **struct proc** — `priority`, `ticks_used`, `enter_seq` + metric fields (`ctime/stime/etime/rtime`).
- **allocproc** — Rule 1: new proc → tail of Q0.
- **scheduler** — Rule 2/6: strict priority, FIFO within a queue via `enter_seq` (round-robin in Q3 falls out naturally). Deadlock-free ascending lock order.
- **mlfq_tick_yield** (trap.c timer hook) — slice accounting, Rule 4 demotion, Rule 2 preemption.
- **sleep / wakeup** — Rule 5: voluntary yield resets slice, re-inserts at tail of same queue.
- **clockintr** — Rule 7: boost all procs to Q0 every 48 ticks.

Files + a reviewable patch are saved:

- `mlfq_changes.patch` (215 insertions, apply with `git apply`)
- Modified `Makefile`, `kernel/{param,proc,defs}.h`, `kernel/proc.c`

+ Write a message...

Claude is AI and can make mistakes. Please double-check responses.

Opus 4.8 Medium

Mlfq trace · LOG

Copy



1	[MLFQ]	t=0	pid=1	q=0	ev=NEW
2	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
3	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
4	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
5	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
6	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
7	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
8	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
9	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
10	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
11	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
12	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
13	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
14	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
15	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
16	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
17	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
18	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
19	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
20	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
21	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
22	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
23	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
24	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
25	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
26	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
27	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
28	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
29	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
30	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
31	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
32	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
33	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
34	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
35	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
36	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
37	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
38	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
39	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
40	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
41	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
42	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
43	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
44	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
45	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
46	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
47	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
48	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
49	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
50	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP



Copy  

Opus 4.8 Medium

Copy

1	[MLFQ]	t=0	pid=1	q=0	ev=NEW
2	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
3	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
4	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
5	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
6	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
7	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
8	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
9	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
10	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
11	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
12	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
13	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
14	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
15	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
16	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
17	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
18	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
19	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
20	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
21	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
22	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
23	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
24	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
25	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
26	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
27	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
28	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
29	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
30	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
31	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
32	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
33	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
34	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
35	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
36	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
37	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
38	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
39	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
40	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
41	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
42	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
43	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
44	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
45	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
46	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
47	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
48	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
49	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
50	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP

The attached full files are the exact verified versions if you'd rather diff/replace directly.

Manual, step by step (7 files)

1. Makefile — three edits.

- After the two `CFLAGS += -I. / -fno-stack-protector` lines, add:

```
make
# Scheduler selection (compile-time). Unset => original round-robin.
ifeq ($(SCHEDULER),MLFQ)
CFLAGS += -D MLFQ
endif
# Optional scheduler tracing for verification / report plots: make ... MLFQDEBUG=1
ifdef MLFQDEBUG
CFLAGS += -D MLFQDEBUG
endif
```

- In the `UPROGS=` list, add a line: `$U/_schedulertest\`

2. kernel/param.h — after `#define USERSTACK 1`:

```
c
#define NMLFQ 4 // number of MLFQ priority queues (0..3)
#define BOOST_INTERVAL 48 // priority boost every this many ticks
```

3. kernel/proc.h — inside `struct proc`, after `char name[16];`:

```
c
int priority; // current queue: 0 (highest) .. NMLFQ-1 (lowest)
int ticks_used; // timer ticks used in current slice at current queue
uint64 enter_seq; // FIFO order of entering current queue (monotonic)
int ctime, stime, etime, rtime; // metrics: create/first-run/exit/cpu ticks
```

4. kernel/defs.h — next to `void yield(void);`:

```
c
void mlfq_tick_yield(void);
void mlfq_boost(void);
```

5. kernel/proc.c — six insertions (see the attached `proc.c` for exact text):

- Globals (after `struct spinlock pid_lock;`): `extern uint ticks;`, the `MLFQLOG` macro,

+ Write a message...

Mlfq trace · LOG

```
1 [MLFQ] t=0 pid=1 q=0 ev=NEW
2 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
3 [MLFQ] t=1 pid=1 q=0 ev=WAKE
4 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
5 [MLFQ] t=1 pid=1 q=0 ev=WAKE
6 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
7 [MLFQ] t=1 pid=1 q=0 ev=WAKE
8 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
9 [MLFQ] t=1 pid=1 q=0 ev=WAKE
10 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
11 [MLFQ] t=1 pid=1 q=0 ev=WAKE
12 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
13 [MLFQ] t=1 pid=1 q=0 ev=WAKE
14 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
15 [MLFQ] t=1 pid=1 q=0 ev=WAKE
16 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
17 [MLFQ] t=1 pid=1 q=0 ev=WAKE
18 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
19 [MLFQ] t=1 pid=1 q=0 ev=WAKE
20 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
21 [MLFQ] t=1 pid=1 q=0 ev=WAKE
22 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
23 [MLFQ] t=1 pid=1 q=0 ev=WAKE
24 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
25 [MLFQ] t=1 pid=1 q=0 ev=WAKE
26 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
27 [MLFQ] t=1 pid=1 q=0 ev=WAKE
28 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
29 [MLFQ] t=1 pid=1 q=0 ev=WAKE
30 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
31 [MLFQ] t=1 pid=1 q=0 ev=WAKE
32 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
33 [MLFQ] t=1 pid=1 q=0 ev=WAKE
34 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
35 [MLFQ] t=1 pid=1 q=0 ev=WAKE
36 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
37 [MLFQ] t=1 pid=1 q=0 ev=WAKE
38 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
39 [MLFQ] t=1 pid=1 q=0 ev=WAKE
40 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
41 [MLFQ] t=1 pid=1 q=0 ev=WAKE
42 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
43 [MLFQ] t=1 pid=1 q=0 ev=WAKE
44 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
45 [MLFQ] t=1 pid=1 q=0 ev=WAKE
46 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
47 [MLFQ] t=1 pid=1 q=0 ev=WAKE
48 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
49 [MLFQ] t=1 pid=1 q=0 ev=WAKE
50 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
```

Implementing MLFQ scheduler in xv6

5. `kernel/proc.c` — six insertions (see the attached `proc.c` for exact text):

- Globals (after `struct spinlock pid_lock`): `extern uint ticks`; the `MLFQLOG` macro, `mlfq_seq`, `mlfq_timeslice[4]={1,4,8,16}`, and the two static helpers `mlfq_next_seq()` / `mlfq_higher_priority_runnable()`.
- `allocproc` (`found`): set metrics + `priority=0`, `ticks_used=0`, `enter_seq=...` (Rule 1).
- `scheduler`: wrap the pick loop in `#ifdef MLFQ` (strict-priority + `enter_seq` FIFO) `#else` original RR `#endif`.
- After `yield()`: add `mlfq_tick_yield()` (slice/demote/preempt) and `mlfq_boost()`.
- `sleep`: on `SLEEPING`, `#ifdef MLFQ` reset `ticks_used=0` (Rule 5).
- `wakeup`: on `SLEEPING→RUNNABLE`, `#ifdef MLFQ` set `enter_seq=mlfq_next_seq()` (Rule 5); `kexit`: `p->etime = ticks`; `procdump`: print the queue columns.

6. `kernel/trap.c` — three edits:

- In `usertrap` and `kerneltrap`, replace `yield()`; at the timer branch with `#ifdef MLFQ` `mlfq_tick_yield()`; `#else` `yield()`; `#endif`.
- In `clockintr`, after releasing `tickslock`, `#ifdef MLFQ` if `(t % BOOST_INTERVAL == 0)` `mlfq_boost()`; `#endif` (capture `t=ticks` under the lock; guard the `uint t`; declaration under `#ifdef MLFQ` too or you'll get an unused-variable error in the RR build — I hit that).

7. `user/schedulertest.c` — new file (attached). Uses `pause(n)` (this variant's name for the sleep syscall) for voluntary yields.

How to reproduce the verification

```
bash
make clean; make qemu SCHEDULER=MLFQ MLFQDEBUG=1
$ schedulertest 3
```

Trace lines look like `[MLFQ] t=48 pid=4 q=0 ev=BOOST`. In my run: CPU-bound child sank `q0→q3` and got boosted back to `q0` at `t=48/96`; I/O-bound child stayed at `q1`. The plain `SCHEDULER=MLFQ` build (no `MLFQDEBUG`) is silent, so it's safe to submit that way.

What this covers vs. the assignment

Done and verified: Build system, §2.1 MLFQ [70] (all 7 rules + `procdump`), and `schedulertest` for §2.3.2.

Still open (need your input before I build them right):

- §2.2 comparison [20] needs a FIFO scheduler — do you have your Homework 2 FIFO, or should I add it as another `SCHEDULER=FIFO` path?
- §2.3 report + plot — the plot must be watermark'd with your IIT username (something@iit.ac.in). Tell me the username and I'll write the Python (parses the `[MLFQ]` trace) +

+ Write a message...

Mlfq trace · LOG

```
1 [MLFQ] t=0 pid=1 q=0 ev=NEW
2 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
3 [MLFQ] t=1 pid=1 q=0 ev=WAKE
4 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
5 [MLFQ] t=1 pid=1 q=0 ev=WAKE
6 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
7 [MLFQ] t=1 pid=1 q=0 ev=WAKE
8 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
9 [MLFQ] t=1 pid=1 q=0 ev=WAKE
10 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
11 [MLFQ] t=1 pid=1 q=0 ev=WAKE
12 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
13 [MLFQ] t=1 pid=1 q=0 ev=WAKE
14 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
15 [MLFQ] t=1 pid=1 q=0 ev=WAKE
16 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
17 [MLFQ] t=1 pid=1 q=0 ev=WAKE
18 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
19 [MLFQ] t=1 pid=1 q=0 ev=WAKE
20 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
21 [MLFQ] t=1 pid=1 q=0 ev=WAKE
22 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
23 [MLFQ] t=1 pid=1 q=0 ev=WAKE
24 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
25 [MLFQ] t=1 pid=1 q=0 ev=WAKE
26 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
27 [MLFQ] t=1 pid=1 q=0 ev=WAKE
28 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
29 [MLFQ] t=1 pid=1 q=0 ev=WAKE
30 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
31 [MLFQ] t=1 pid=1 q=0 ev=WAKE
32 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
33 [MLFQ] t=1 pid=1 q=0 ev=WAKE
34 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
35 [MLFQ] t=1 pid=1 q=0 ev=WAKE
36 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
37 [MLFQ] t=1 pid=1 q=0 ev=WAKE
38 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
39 [MLFQ] t=1 pid=1 q=0 ev=WAKE
40 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
41 [MLFQ] t=1 pid=1 q=0 ev=WAKE
42 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
43 [MLFQ] t=1 pid=1 q=0 ev=WAKE
44 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
45 [MLFQ] t=1 pid=1 q=0 ev=WAKE
46 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
47 [MLFQ] t=1 pid=1 q=0 ev=WAKE
48 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
49 [MLFQ] t=1 pid=1 q=0 ev=WAKE
50 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
```

Mlfg trace LOG

Copy   

1	[MLFQ]	t=0	pid=1	q=0	ev=NEW
2	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
3	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
4	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
5	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
6	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
7	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
8	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
9	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
10	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
11	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
12	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
13	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
14	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
15	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
16	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
17	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
18	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
19	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
20	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
21	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
22	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
23	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
24	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
25	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
26	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
27	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
28	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
29	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
30	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
31	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
32	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
33	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
34	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
35	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
36	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
37	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
38	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
39	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
40	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
41	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
42	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
43	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
44	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
45	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
46	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
47	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
48	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
49	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
50	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP

1. **\$2.2 comparison [20]** needs a **FIFO** scheduler — do you have your Homework 2 FIFO, or should I add it as another **SCHEDULER=FIFO** path?
2. **\$2.3 report + plot** — the plot must be watermarked with your IIIT username (`something@iiit.ac.in`). Tell me the username and I'll write the Python (parses the `[MLFQ] ... trace`) + the report.

One caveat for your viva: I represent the four queues implicitly (`priority + enter_seq`) rather than as explicit arrays. Flag it if your TA wants literal array-queues and I'll refactor.

Want me to proceed with FIFO + the report/plot next?

Download

Download

Download

Download

Download

Download

Download

Download

Download all



+ Write a message...

Claude is AI and can make mistakes. Please double-check responses.

Opus 4.8 Medium

In section 2.2 of the report (xv6), you are required to watermark the plots that you create with the part before @ in your IIT email (username from username@{students,research}.iit.ac.in)

```
plt.text(
    0.95, 0.95, "username",
    ha='right', va='top',
    transform=plt.gca().transAxes,
    fontsize=10, color="gray", alpha=0.7
)
```

You are also required to submit the python code used for plot generation.

Preparing to build a comparison chart pending missing metrics. >

For the plots you need two things: the plotting code (watermarked, submittable) and real trace data to plot. I have the trace data verified. Let me build tested plotting code that produces the watermarked MLFQ timeline, and run it so I'm handing you code that actually works — not untested code.

First, matplotlib + a clean trace capture:

Ran 2 commands, created a file, read a file >

The plot works and the watermark renders exactly as specified (top-right, gray). But it's not readable: init/sh boost events stretch the x-axis out to tick 1400 while the actual workload sits at ticks ~260~360. Let me fix the script to zoom to the workload window:

Edited a file, ran a command, read a file >

That's a clean, submission-quality plot. It clearly shows all the MLFQ dynamics:

- **pid 4 (blue, CPU-bound):** sinks $q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_3$, gets boosted back to q_0 at the dotted boost lines (~t288, ~t336), then sinks again
- **pid 5 (orange, mixed):** oscillates $q_0 \leftrightarrow q_2$
- **pid 6 (green, I/O-bound):** stays at q_1 , never sinks — because it yields before using its slice
- Dotted vertical lines = the 48-tick boosts; watermark top-right exactly per your snippet

Let me package the code and sample output:

Ran a command, shared files >

plot.py is tested and produces the watermarked timeline you see above. Here's everything you

+ Write a message...

MLfq trace LOG

```
1 [MLFQ] t=0 pid=1 q=0 ev=NEW
2 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
3 [MLFQ] t=1 pid=1 q=0 ev=WAKE
4 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
5 [MLFQ] t=1 pid=1 q=0 ev=WAKE
6 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
7 [MLFQ] t=1 pid=1 q=0 ev=WAKE
8 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
9 [MLFQ] t=1 pid=1 q=0 ev=WAKE
10 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
11 [MLFQ] t=1 pid=1 q=0 ev=WAKE
12 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
13 [MLFQ] t=1 pid=1 q=0 ev=WAKE
14 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
15 [MLFQ] t=1 pid=1 q=0 ev=WAKE
16 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
17 [MLFQ] t=1 pid=1 q=0 ev=WAKE
18 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
19 [MLFQ] t=1 pid=1 q=0 ev=WAKE
20 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
21 [MLFQ] t=1 pid=1 q=0 ev=WAKE
22 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
23 [MLFQ] t=1 pid=1 q=0 ev=WAKE
24 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
25 [MLFQ] t=1 pid=1 q=0 ev=WAKE
26 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
27 [MLFQ] t=1 pid=1 q=0 ev=WAKE
28 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
29 [MLFQ] t=1 pid=1 q=0 ev=WAKE
30 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
31 [MLFQ] t=1 pid=1 q=0 ev=WAKE
32 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
33 [MLFQ] t=1 pid=1 q=0 ev=WAKE
34 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
35 [MLFQ] t=1 pid=1 q=0 ev=WAKE
36 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
37 [MLFQ] t=1 pid=1 q=0 ev=WAKE
38 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
39 [MLFQ] t=1 pid=1 q=0 ev=WAKE
40 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
41 [MLFQ] t=1 pid=1 q=0 ev=WAKE
42 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
43 [MLFQ] t=1 pid=1 q=0 ev=WAKE
44 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
45 [MLFQ] t=1 pid=1 q=0 ev=WAKE
46 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
47 [MLFQ] t=1 pid=1 q=0 ev=WAKE
48 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
49 [MLFQ] t=1 pid=1 q=0 ev=WAKE
50 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
```

`plot.py` is tested and produces the watermarked timeline you see above. Here's everything you need to use it and submit it.

What you must do

1. Set your username — one line at the top of `plot.py`:

```
python
USERNAME = "your_username" # <-- the part before @ in your IIIT email
```

Right now it says `your_username` (visible in the sample). I don't know your IIIT username — your GitHub handle `Radzzz06` isn't the same thing — so tell me and I'll bake it in, or just edit that line.

2. Enable trace capture. `plot.py` reads the kernel's `[MLFQ]` ... trace, but your repo currently has logging switched off. Two small things (the first you need anyway to make MLFQ compile at all):

- The `MLFQLOG` macro fix I gave you last message (in `kernel/proc.c`).
- Add the 3-line `MLFQDEBUG` toggle to your `Makefile`, right after the `SCHEDULER=MLFQ` block:

```
make
ifdef MLFQDEBUG
CFLAGS += -D MLFQDEBUG
endif
```

3. Capture the trace and plot:

```
bash
make clean && make qemu SCHEDULER=MLFQ MLFQDEBUG=1
# inside xv6:
$ scheduler_test 3
# quit: Ctrl-A then X
# keep only trace lines from the saved console output:
grep '^[MLFQ\]' console_output.txt > mlfq_trace.log
python3 plot.py mlfq_trace.log
```

This writes `mlfq_timeline.png`. For submission, rebuild **without** `MLFQDEBUG=1` so the graded binary stays silent.

The second plot (§2.2 comparison) — blocked on FIFO

`plot.py` also has `plot_comparison()`, which draws a watermarked FIFO-vs-RR-vs-MLFQ bar chart from a `metrics.csv`:

```
scheduler,turnaround,waiting,response
FIFO,120.5,80.2,15.0
```

Mlfq trace · LOG

```
1 [MLFQ] t=0 pid=1 q=0 ev=NEW
2 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
3 [MLFQ] t=1 pid=1 q=0 ev=WAKE
4 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
5 [MLFQ] t=1 pid=1 q=0 ev=WAKE
6 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
7 [MLFQ] t=1 pid=1 q=0 ev=WAKE
8 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
9 [MLFQ] t=1 pid=1 q=0 ev=WAKE
10 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
11 [MLFQ] t=1 pid=1 q=0 ev=WAKE
12 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
13 [MLFQ] t=1 pid=1 q=0 ev=WAKE
14 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
15 [MLFQ] t=1 pid=1 q=0 ev=WAKE
16 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
17 [MLFQ] t=1 pid=1 q=0 ev=WAKE
18 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
19 [MLFQ] t=1 pid=1 q=0 ev=WAKE
20 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
21 [MLFQ] t=1 pid=1 q=0 ev=WAKE
22 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
23 [MLFQ] t=1 pid=1 q=0 ev=WAKE
24 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
25 [MLFQ] t=1 pid=1 q=0 ev=WAKE
26 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
27 [MLFQ] t=1 pid=1 q=0 ev=WAKE
28 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
29 [MLFQ] t=1 pid=1 q=0 ev=WAKE
30 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
31 [MLFQ] t=1 pid=1 q=0 ev=WAKE
32 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
33 [MLFQ] t=1 pid=1 q=0 ev=WAKE
34 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
35 [MLFQ] t=1 pid=1 q=0 ev=WAKE
36 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
37 [MLFQ] t=1 pid=1 q=0 ev=WAKE
38 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
39 [MLFQ] t=1 pid=1 q=0 ev=WAKE
40 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
41 [MLFQ] t=1 pid=1 q=0 ev=WAKE
42 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
43 [MLFQ] t=1 pid=1 q=0 ev=WAKE
44 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
45 [MLFQ] t=1 pid=1 q=0 ev=WAKE
46 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
47 [MLFQ] t=1 pid=1 q=0 ev=WAKE
48 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
49 [MLFQ] t=1 pid=1 q=0 ev=WAKE
50 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
```



Implementing MLFQ scheduler in xv6

plot.py also has `plot_comparison()`, which draws a watermarked FIFO-vs-RR-vs-MLFQ bar chart from a `metrics.csv`:

```
scheduler,turnaround,waiting,response
FIFO,120.5,80.2,15.0
RR,95.1,55.3,4.2
MLFQ,88.7,50.1,3.1
```

I deliberately did not invent those numbers — putting fabricated data in your report would be academic dishonesty. To get real numbers you need (a) the **FIFO** scheduler and (b) a way to measure `turnaround/waiting/response` (I already added `ctime/stime/etime/rtime` fields; they just need a `waitx`-style syscall to read them out). Once that's in, you run the same workload under `SCHEDULER=FIFO`, `` (RR default), and `SCHEDULER=MLFQ`, fill `metrics.csv`, and run `python3 plot.py mlfq_trace.log metrics.csv`.

Files delivered

- `plot.py` — watermarked, tested plotting code (submit this)
- `mlfq_timeline.png` — sample output
- `mlfq_trace.log` — the sample trace it was built from
- (from before) the `MLFQLOG` fix in `proc_fixed.c`

Want me to implement the **FIFO** scheduler + the `waitx` metrics syscall now so you can generate the real comparison numbers and complete §2.2? That's the last missing piece. And give me your username so the watermark is final.

 Plot
PY

Download

 Mlfq timeline
Image · PNG

Download

 Mlfq trace
LOG

Download

Download all

3 hours ago



+ Write a message...



Mlfq trace · LOG

```
1 [MLFQ] t=0 pid=1 q=0 ev=NEW
2 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
3 [MLFQ] t=1 pid=1 q=0 ev=WAKE
4 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
5 [MLFQ] t=1 pid=1 q=0 ev=WAKE
6 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
7 [MLFQ] t=1 pid=1 q=0 ev=WAKE
8 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
9 [MLFQ] t=1 pid=1 q=0 ev=WAKE
10 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
11 [MLFQ] t=1 pid=1 q=0 ev=WAKE
12 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
13 [MLFQ] t=1 pid=1 q=0 ev=WAKE
14 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
15 [MLFQ] t=1 pid=1 q=0 ev=WAKE
16 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
17 [MLFQ] t=1 pid=1 q=0 ev=WAKE
18 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
19 [MLFQ] t=1 pid=1 q=0 ev=WAKE
20 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
21 [MLFQ] t=1 pid=1 q=0 ev=WAKE
22 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
23 [MLFQ] t=1 pid=1 q=0 ev=WAKE
24 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
25 [MLFQ] t=1 pid=1 q=0 ev=WAKE
26 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
27 [MLFQ] t=1 pid=1 q=0 ev=WAKE
28 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
29 [MLFQ] t=1 pid=1 q=0 ev=WAKE
30 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
31 [MLFQ] t=1 pid=1 q=0 ev=WAKE
32 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
33 [MLFQ] t=1 pid=1 q=0 ev=WAKE
34 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
35 [MLFQ] t=1 pid=1 q=0 ev=WAKE
36 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
37 [MLFQ] t=1 pid=1 q=0 ev=WAKE
38 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
39 [MLFQ] t=1 pid=1 q=0 ev=WAKE
40 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
41 [MLFQ] t=1 pid=1 q=0 ev=WAKE
42 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
43 [MLFQ] t=1 pid=1 q=0 ev=WAKE
44 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
45 [MLFQ] t=1 pid=1 q=0 ev=WAKE
46 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
47 [MLFQ] t=1 pid=1 q=0 ev=WAKE
48 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
49 [MLFQ] t=1 pid=1 q=0 ev=WAKE
50 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
```



- RS RE   



In **Makefile**, change your scheduler block to also understand **LOG=1**:

```
make

# Scheduler selection (compile-time), else remains round robin
ifeq ($(SCHEDULER),MLFQ)
CFLAGS += -D MLFQ
ifeq ($(LOG),1)
CFLAGS += -D MLFQ_LOG
endif
endif
```

Now make `clean` && make `qemu SCHEDULER=MLFQ CPUS=1` compiles and runs (RR-free MLFQ), and `LOG=1` turns on the plot log. Use `CPUS=1` — a single CPU makes the timeline clean and correct.

Fix 2 — make the comparison possible (2.2 / 2.3.3)

The comparison needs Turnaround, Waiting, Response for **FIFO, RR, MLFQ**. Three small gaps in your kernel:

(a) `rtime` is only counted in MLFQ. Move CPU-time counting so it works in every scheduler. In `kernel/proc.c` add a helper:

```
c
// Count one CPU tick for whatever process is RUNNING (any scheduler).
void
update_time(void)
{
    struct proc *p;
    for (p = proc; p < &proc[NPROC]; p++) {
        acquire(&p->lock);
        if (p->state == RUNNING)
            p->rtime++;
        release(&p->lock);
    }
}
```

Declare it in `kernel/defs.h` next to the other proc functions: `void update_time(void);`
 Then delete the `p->rtime++;` line inside `mlfq_tick_yield()` (so you don't double-count),
 and in `kernel/trap.c`'s `clockintr()`, call `update_time()`; right after
`release(&tickslock);` (outside any `#ifdef MLFQ`).

(b) Response time isn't recorded in RR. In scheduler()'s #else (RR) branch, just before p->state = RUNNING; add:

+ Write a message...



kernel/trap.c — FIFO is non-preemptive, so it must *not* yield on a timer tick. Change both tick spots (usertrap and kerneltrap) from the two way `#ifdef MLFQ / #else` to:



Claude

New

Projects

Artifacts

Scheduled

Customize

Projects

Comeback

Chats and tasks

◊

xv6 MLFQ scheduling

Investigating an issue

Matplotlib import error

Multi-level priority queue

Implementing MLFQ scheduling

Git push permission denied

Setting up xv6 MLFQ scheduling

Missing code in c-shell

Running xv6 offline on Raspberry Pi

Diverged git branches

GitHub project flowchart

Independence vs. uncertainty

MLFQ scheduler loop

Beginner-friendly xv6

Character class negation

Workout plan for back pain

Running xv6-riscv on Raspberry Pi

Dangling-Else resolution

Beginner-friendly code

Beginner-friendly code

View all conversations

Design

RS Re ↵ 🔍 📄

📄 ↵

claude.ai

🔄

📄 + 📄

🔍 ↵

/6 MLFQ scheduling project setup

📄 Share

kernel/trap.c — FIFO is non-preemptive, so it must *not* yield on a timer tick. Change both tick spots (usertrap and kerneltrap) from the two-way `#ifdef MLFQ / #else` to:

```
c
#ifdef MLFQ
    mlq_tick_yield();
#elif defined(FIFO)
    ;
#else
    yield();
#endif
```

kernel/proc.c — turn the `#ifdef MLFQ ... #else ... (RR) ... #endif` in `scheduler()` into `#if defined(MLFQ) ... #elif defined(FIFO) ... #else ... #endif`, with this FIFO block (picks the earliest-arrived RUNNABLE process and runs it):

```
c
#ifdef FIFO
while(1){
    intr_on(); intr_off();
    struct proc *best = 0;
    for (p = proc; p < &proc[NPROC]; p++){
        acquire(&p->lock);
        if (p->state == RUNNABLE){
            if (best == 0){ best = p; continue; }
            if (p->ctime < best->ctime){ release(&best->lock); best = p; }
            else release(&p->lock);
        } else release(&p->lock);
    }
    if (best){
        if (best->stime < 0) best->stime = ticks;
        best->state = RUNNING;
        c->proc = best;
        switch(&c->context, &best->context);
        mycpu()->intena = 0;
        c->proc = 0;
        release(&best->lock);
    } else {
        asm volatile("wfi");
    }
}
#endif
```

+ Write a message...

🔍 🗣️ ↵

Claude is AI and can make mistakes. Please double-check responses.

Opus 4.8 High

- RS R₂   

[illegible]

I didn't understand Fix 3 — add the FIFO scheduler (needed for the 3-way comparison) and later, please give step by step instruction it is just so ugly thsi way

Explaining a FIFO scheduler in clear beginner-friendly steps.

Let me pull up the exact current text of your `scheduler()` and the two tick spots in `trap.c`, so I can show you precise before → after edits instead of vague snippets.

Ran a command >

Totally fair — that section was cramped. Let me slow it right down. First the idea, then three clean steps.

What FIFO actually is (the mental model)

FIFO = First In, First Out, and it's **non-preemptive**. Whichever process *arrived first* runs to completion (or until it sleeps for I/O) before anyone else gets a turn. Think of a single queue at a counter: the first person is served fully, then the next.

Compare the three:

- **RR** kicks the running process off every timer tick and rotates to the next. (Preemptive).
- **MLFQ** has 4 queues + time slices, demotes CPU hogs, boosts every 48 ticks.
- **FIFO** does neither — pick the earliest arrival, let it run until it finishes. No slicing, no rotating.

So adding FIFO needs edits in two places, and it helps to know why each one exists:

- `scheduler()` in `kernel/proc.c` decides **which** process to pick → for FIFO, pick the one with the smallest `ctime` (earliest arrival).
- `kernel/trap.c` decides, on each timer tick, **whether to kick the running process off** → for FIFO the answer is *no, never* (that's what makes it non-preemptive).

That's the whole thing. Now the steps.

Step 1 — Makefile: add a **FIFO** flag

Find your MLFQ block in **Makefile**:

```
make

ifeq ($ (SCHEDULER), MLFQ)
CFLAGS += -D MLFQ
```

+ Write a message...



Claude

New

Projects

Artifacts

Scheduled

Customize

Projects

Comeback

Chats and tasks

xv6 MLFQ scheduling

Investigating an issue

Matplotlib import error

Multi-level priority queue

Implementing MLFQ scheduling

Git push permission denied

Setting up xv6 MLFQ scheduling

Missing code in c-shell

Running xv6 offline on a Raspberry Pi

Diverged git branches

GitHub project flowchart

Independence vs. interdependence

MLFQ scheduler loop

Beginner-friendly xv6

Character class negation

Workout plan for back pain

Running xv6-riscv on a Raspberry Pi

Dangling-Else resolution

Beginner-friendly code

Beginner-friendly code

View all conversations

Design

RS Re 🔍 📄

6 MLFQ scheduling project setup

make

ifeq (\$(SCHEDULER),MLFQ)

CFLAGS += -D MLFQ

ifeq (\$(LOG),1)

CFLAGS += -D MLFQ_LOG

endif

endif

Add this right below it:

make

ifeq (\$(SCHEDULER),FIFO)

CFLAGS += -D FIFO

endif

Now `make qemu SCHEDULER=FIFO` defines the `FIFO` macro; the C code below reacts to it.

Step 2 — `kernel/proc.c`: teach `scheduler()` the FIFO pick

Your `scheduler()` currently has this shape (MLFQ, then a plain `#else` for round-robin):

c

#ifdef MLFQ

... MLFQ loop ...

#else

... round-robin loop ...

#endif

You only change one line: the `#else` that sits right after the MLFQ loop's closing brace. Find that exact line:

c

}

}

#else

while(1) {

// The most recent process to run may have had interrupts

Replace only that `#else` with the FIFO block below plus a fresh `#else`:

c

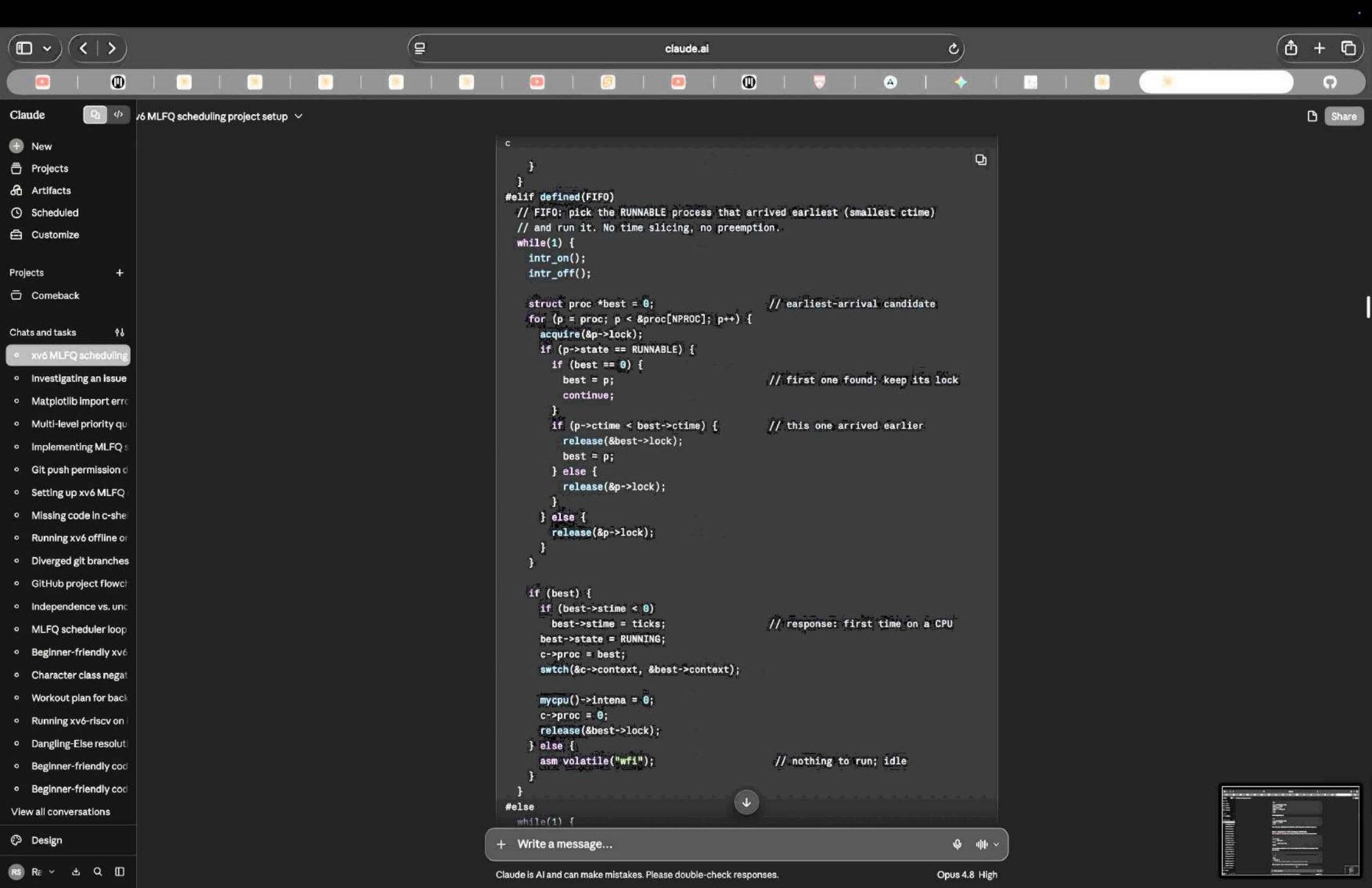
+ Write a message...

🔊 🔊 🔊

Claude is AI and can make mistakes. Please double-check responses.

Opus 4.8 High





Notice the FIFO loop is almost identical to the MLFQ one; the *only* real difference is the pick rule: MLFQ compares `priority / enter_seq`, FIFO compares `ctime`.

There are two identical-looking tick spots. One is in `usertrap()`, the other in `kerneltrap()`. Each looks like this:

Change **both** of them to add a FIFO branch that does nothing:

That bare `;` is a do-nothing statement — on FIFO the timer tick fires, but the process is *not* kicked off, so it keeps running until it exits or sleeps. That's exactly the non-preemptive behaviour.

It should boot to the `$` prompt like normal. Run `schedulertest` — under FIFO you'll see each child finish more or less in the order it started, rather than being interleaved the way RR/MLFQ do it. That difference is exactly what your comparison table in 2.3.3 is capturing.

Claude

New

Projects

Artifacts

Scheduled

Customize

Projects

Comeback

Chats and tasks

xv6 MLFQ scheduling

Investigating an issue

Matplotlib import error

Multi-level priority queue

Implementing MLFQ

Git push permission error

Setting up xv6 MLFQ

Missing code in c-shell

Running xv6 offline on

Diverged git branches

GitHub project flowchart

Independence vs. un

MLFQ scheduler loop

Beginner-friendly xv6

Character class negat

Workout plan for back

Running xv6-riscv on

Dangling-Else resolut

Beginner-friendly cod

Beginner-friendly cod

View all conversations

Design

RS Re Q

6 MLFQ scheduling project setup

Step 4 — build and check

```
make clean
make qemu SCHEDULER=FIFO CPUS=1
```

It should boot to the `$` prompt like normal. Run `schedulertest` — under FIFO you'll see each child finish more or less in the order it started, rather than being interleaved the way RR/MLFQ do it. That difference is exactly what your comparison table in 2.3.3 is capturing.

Two things worth knowing so the TA viva goes smoothly:

FIFO doesn't hang the shell even though it's "run to completion," because processes like `sh` and `init` spend most of their time *asleep* waiting for input — sleeping is a voluntary yield, so FIFO happily moves on. It only "runs to completion" among processes that actually want the CPU.

And if you already wrote a FIFO scheduler in Homework 2, use that one instead and just make sure the `Makefile` flag and the `trap.c` no-preempt branch match — the graders expect the HW2 FIFO.

Want me to do the same slow walkthrough for Fix 2 (the metrics/`rtime` changes), since that one touched three files too?

3 hours ago

```
riscv64-unknown-elf-ld -z max-page-size=4096 -T kernel/kernel.ld -o kernel/kernel
kernel/entry.o kernel/start.o kernel/console.o kernel/printk.o kernel/uart.o
kernel/kalloc.o kernel/spinlock.o kernel/string.o kernel/main.o kernel/vm.o
kernel/proc.o kernel/swtch.o kernel/trampoline.o kernel/trap.o kernel/syscall.o
kernel/sysproc.o kernel/bio.o kernel/fs.o kernel/log.o kernel/sleeplock.o kernel/file.o
kernel/pipe.o kernel/exec.o kernel/sysfile.o kernel/kernelvec.o kernel/pllc.o
kernel/virtio_disk.o
riscv64-unknown-elf-ld: warning: kernel/kernel has a LOAD segment with RWX
permissions
riscv64-unknown-elf-ld: kernel/trap.o: in function 'clockintr':
./kernel/trap.c:188:(.text+0x108): undefined reference to 'update_time'
make: *** [kernel/kernel] Error 1
is what is printed
```

Pinpointing a missing function outside an ifdef block.

That error is a **linker error**, and it's a common, **easy** one. Quick translation of what the machine is telling you:

Write a message...

Claude is AI and can make mistakes. Please double-check responses.

Opus 4.8 High

xv6 MLFQ schedulin...

Share

- [View all conversations](#)

```
void update_time(void);
```

Run this in your `xv6/` folder — it tells you whether the body is really there and whether it's trapped inside an `#ifdef`:

You want to see it in both files. In `proc.c` you should see two lines: the `void` line and the `update_time(void)` line of the definition — and they should be nowhere near an `#ifdef` MLFQ.

```
make clean
make qemu CPUS=1
```

One thing while you're in there: make sure you also **deleted** the old `p->rtime++;` line inside `mlfq_tick_yield()`. That's not what's breaking the build, but if you leave it in, MLFQ runs will count CPU time twice and your comparison numbers for MLFQ will come out wrong.

3 hours ago

Working through a multi-part series of code fixes.

Ran a command >

What the three metrics mean (one concrete example)

+ Write a message...



Claude

New

Projects

Artifacts

Scheduled

Customize

Projects

Comeback

Chats and tasks

xv6 MLFQ scheduling

Investigating an issue

Matplotlib import error

Multi-level priority queue

Implementing MLFQ

Git push permission denied

Setting up xv6 MLFQ

Missing code in c-shell

Running xv6 offline on Raspberry Pi

Diverged git branches

GitHub project flowchart

Independence vs. unity

MLFQ scheduler loop

Beginner-friendly xv6

Character class negation

Workout plan for back pain

Running xv6-riscv on Raspberry Pi

Dangling-Else resolution

Beginner-friendly code

Beginner-friendly code

View all conversations

Design

RS Re ↵ 🔍 📄

6 MLFQ scheduling project setup

Share

Say a child process:

arrives at tick 3 → ctime = 3

first gets the CPU at tick 5 → stime = 5

finishes at tick 61 → etime = 61

actually spent 40 ticks running → rtime = 40

Then:

Response = stime - ctime = 5 - 3 = 2 (how long until it first ran)

Turnaround = etime - ctime = 61 - 3 = 58 (total lifetime)

Waiting = Turnaround - rtime = 58 - 40 = 18 (time it existed but was not on the CPU)

Your kernel already records all four raw numbers. The two edits below just make sure they're recorded in every scheduler and printed where you can read them.

Edit 1 — record response time in the RR path

MLFQ and your new FIFO branch already set stime. The plain round-robin branch doesn't, so without this, RR's response numbers would be garbage (-1).

In kernel/proc.c, in the #else (round-robin) branch of scheduler(), find:

c

```
if (p->state == RUNNABLE) {
    // Switch to chosen process. It is the process's job
    // to release its lock and then reacquire it
    // before jumping back to us.
    p->state = RUNNING;
```

Add one line just before p->state = RUNNING;:

c

```
if (p->stime < 0) p->stime = ticks; // response: first time on a CPU
p->state = RUNNING;
```

Now all three schedulers record response time the same way.

Edit 2 — print the numbers when a process exits

In kernel/proc.c, inside kexit(), find the line you already have:

c

```
p->stime = ticks; // completion time ( turnaround = etime - stime )
```

+ Write a message...

🗣️ 🔊 🔇

Claude is AI and can make mistakes. Please double-check responses.

Opus 4.8 High



Opus 4.8 High

```
python3 perf_table.py fifo_perf.log rr_perf.log mlfq_perf.log
```

To see it working, I fed the script some **made-up** logs — here's what it printed (these are *not* your real numbers, just a demo of the format):

Notice the shape even in fake data: FIFO's response time is awful (later jobs wait for the first to finish), while RR and MLFQ respond almost instantly — that's the exact trade-off your 5-8 sentence discussion should explain.

```
python3 perf_table.py --children-only fifo_perf.log rr_perf.log mlfq_perf.log
```

That completes all the kernel work. Where you stand now:

The only things left are *running* the captures and filling the blanks in the report, then exporting it to PDF and logging the AI-generated snippets in `AI-usage.pdf`.

Claude

New

Projects

Artifacts

Scheduled

Customize

Projects

Comeback

Chats and tasks

xv6 MLFQ scheduling

Investigating an issue

Matplotlib import error

Multi-level priority queue

Implementing MLFQ scheduler

Git push permission error

Setting up xv6 MLFQ

Missing code in c-shell

Running xv6 offline on Mac

Diverged git branches

GitHub project flowchart

Independence vs. Unix

MLFQ scheduler loop

Beginner-friendly xv6

Character class negation

Workout plan for back

Running xv6-riscv on Mac

Dangling-Else resolution

Beginner-friendly code

Beginner-friendly code

View all conversations

Design

RS Re ↕ 🔍 📄

6 MLFQ scheduling project setup

python3 perf_table.py fifo_perf.log rr_perf.log mlfq_perf.log
/opt/homebrew/Cellar/python@3.14/3.14.7/Frameworks/Python.framework/Versions/
3.14/Resources/Python.app/Contents/MacOS/Python: can't open file
'/Users/radhikasahu/Desktop/Sem_3/OSN/mini-project1/xv6/perf_table.py': [Errno 2]
No such file or director now what tp do?

Diagnosing a missing file error in a script run.

That error just means Python is looking for `perf_table.py` in your `xv6/` folder and it isn't there yet — the file is still wherever your browser saved it (on a Mac that's almost always `~/Downloads`). Nothing is broken; the script just needs to sit next to the log files it reads.

Here's the fix, step by step.

Step 1 — make sure you actually downloaded it. In this chat, click the `perf_table.py` file card and choose download/save. On a Mac it lands in `~/Downloads/perf_table.py`. Do the same for `plot_mlfq.py` and `report.md` while you're at it.

Step 2 — move it into your `xv6` folder. Open Terminal and run:

```
mv ~/Downloads/perf_table.py /Users/radhikasahu/Desktop/Sem_3/OSN/mini-project1/xv6/  
mv ~/Downloads/plot_mlfq.py /Users/radhikasahu/Desktop/Sem_3/OSN/mini-project1/xv6/
```

(If the browser renamed the file to something like `perf_table-1.py` or `perf table.py`, use that exact name in the `mv` command — type `ls ~/Downloads/perf*` to see what it's actually called.)

Step 3 — go into the `xv6` folder and check what's there:

```
cd /Users/radhikasahu/Desktop/Sem_3/OSN/mini-project1/xv6  
ls perf_table.py *.log
```

You want to see `perf_table.py` and your three log files (`fifo_perf.log`, `rr_perf.log`, `mlfq_perf.log`).

Step 4 — run it:

```
python3 perf_table.py fifo_perf.log rr_perf.log mlfq_perf.log
```

One thing to watch for in Step 3: if `ls *.log` says "No such file or directory," it means you

+ Write a message...

Claude is AI and can make mistakes. Please double-check responses.

Opus 4.8 High

xv6 MLFQ scheduling...

Share



One thing to watch for in Step 3: if `ls *.log` says "No such file or directory," it means you haven't captured the run logs yet — the script has nothing to read. In that case do the capture runs first:

```
make clean && make qemu SCHEDULER=FIFO PERF=1 CPUS=1 | tee fifo_perf.log
# type schedulertest at the $ prompt, let it finish, then press Ctrl-a then x to
quit
make clean && make qemu PERF=1 CPUS=1 | tee rr_perf.log
make clean && make qemu SCHEDULER=MLFQ PERF=1 CPUS=1 | tee mlfq_perf.log
```

Then run the Step 4 command.

If it still complains after moving the file, paste the output of `ls perf_table.py *.log` and I'll tell you exactly what's missing.

see now I realised that maybe make clean si deleting these files so created?

2 hours ago

Ran 2 commands >

You're exactly right — nice catch. Look at the very first line of the `clean` target:

```
make
clean:
    rm -f *.tex *.dvi *.idx *.aux *.log *.ind *.ilg \
```

That `*.log` is meant for LaTeX log files, but the wildcard doesn't care — it matches any file ending in `.log`, so it wipes your `fifo_perf.log`, `rr_perf.log`, `mlfq_perf.log`, and `mlfq.log` too.

Here's the sneaky part that explains why you ended up with only one log. Each capture command is `make clean && make qemu ... | tee X.log`. The `clean` runs *first*, then `tee` creates the new log — so that run's log is fine. But the *next* command's `make clean` then deletes it:

1. `make clean && ... | tee fifo_perf.log` → creates `fifo_perf.log` ✓
2. `make clean && ... | tee rr_perf.log` → clean deletes `fifo_perf.log`, creates `rr_perf.log`
3. `make clean && ... | tee mlfq_perf.log` → clean deletes `rr_perf.log`, creates `mlfq_perf.log`

+ Write a message...

Claude is AI and can make mistakes. Please double-check responses.

Opus 4.8 High



Claude

New

Projects

Artifacts

Scheduled

Customize

Projects

Comeback

Chats and tasks

xv6 MLFQ scheduling

Investigating an issue

Matplotlib import error

Multi-level priority queue

Implementing MLFQ

Git push permission error

Setting up xv6 MLFQ

Missing code in c-shell

Running xv6 offline on

Diverged git branches

GitHub project flowchart

Independence vs. unification

MLFQ scheduler loop

Beginner-friendly xv6

Character class negation

Workout plan for back

Running xv6-riscv on

Dangling-Else resolution

Beginner-friendly code

Beginner-friendly code

View all conversations

Design

RS Re Download Search Window

6 MLFQ scheduling project setup

1. `make clean && ... | tee fifo_perf.log` → creates `fifo_perf.log` ✓

2. `make clean && ... | tee rr_perf.log` → clean deletes `fifo_perf.log`, creates `rr_perf.log`

3. `make clean && ... | tee mlfq_perf.log` → clean deletes `rr_perf.log`, creates `mlfq_perf.log`

At the end only `mlfq_perf.log` survives. That's the whole mystery.

(Your `.py` scripts were never at risk — there's no `*.py` in that `clean` rule. The earlier "can't open `perf_table.py`" was just that the file hadn't been moved into the folder yet, a separate thing.)

The fix: name your logs with a non-`.log` extension

The scripts don't care about the extension — they read whatever filename you hand them. Just use `.txt` and `make clean` leaves them alone:

```
make clean && make qemu SCHEDULER=FIFO PERF=1 CPUS=1 | tee fifo_perf.txt
# type schedulertest, let it finish, Ctrl-a then x to quit
make clean && make qemu PERF=1 CPUS=1 | tee rr_perf.txt
make clean && make qemu SCHEDULER=MLFQ PERF=1 CPUS=1 | tee mlfq_perf.txt

python3 perf_table.py fifo_perf.txt rr_perf.txt mlfq_perf.txt
```

And for the plot log:

```
make clean && make qemu SCHEDULER=MLFQ LOG=1 CPUS=1 | tee mlfq.txt
python3 plot_mlfq.py mlfq.txt
```

You still need the `make clean` before each run — switching the `SCHEDULER` flag only takes effect after a clean rebuild, otherwise `make` reuses the old `.o` files and silently runs the wrong scheduler. So keep the clean; just keep your captures out of its blast radius by using `.txt`.

One more safety habit: after all three runs, check they're all still there before you parse:

```
ls -la *perf*.txt
```

You should see all three files with non-zero sizes. If one is 0 bytes, that run didn't capture anything — rerun just that one.

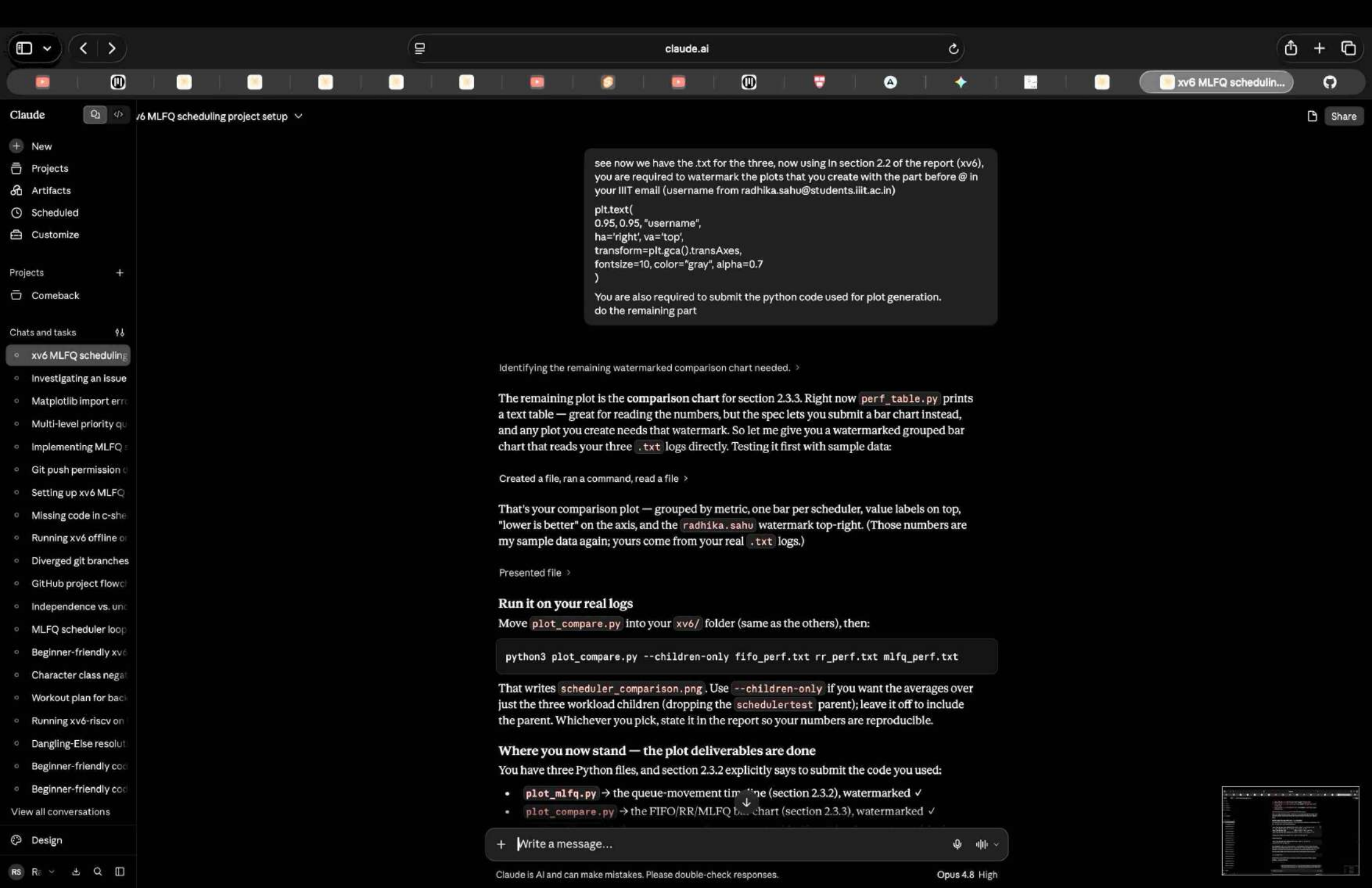
see now we have the `.txt` for the three, now using in section 2.2 of the report (xv6), you are required to watermark plots that you create with the part before @ in your IIT email (username from `radhika.sahu@students.iit.ac.in`)

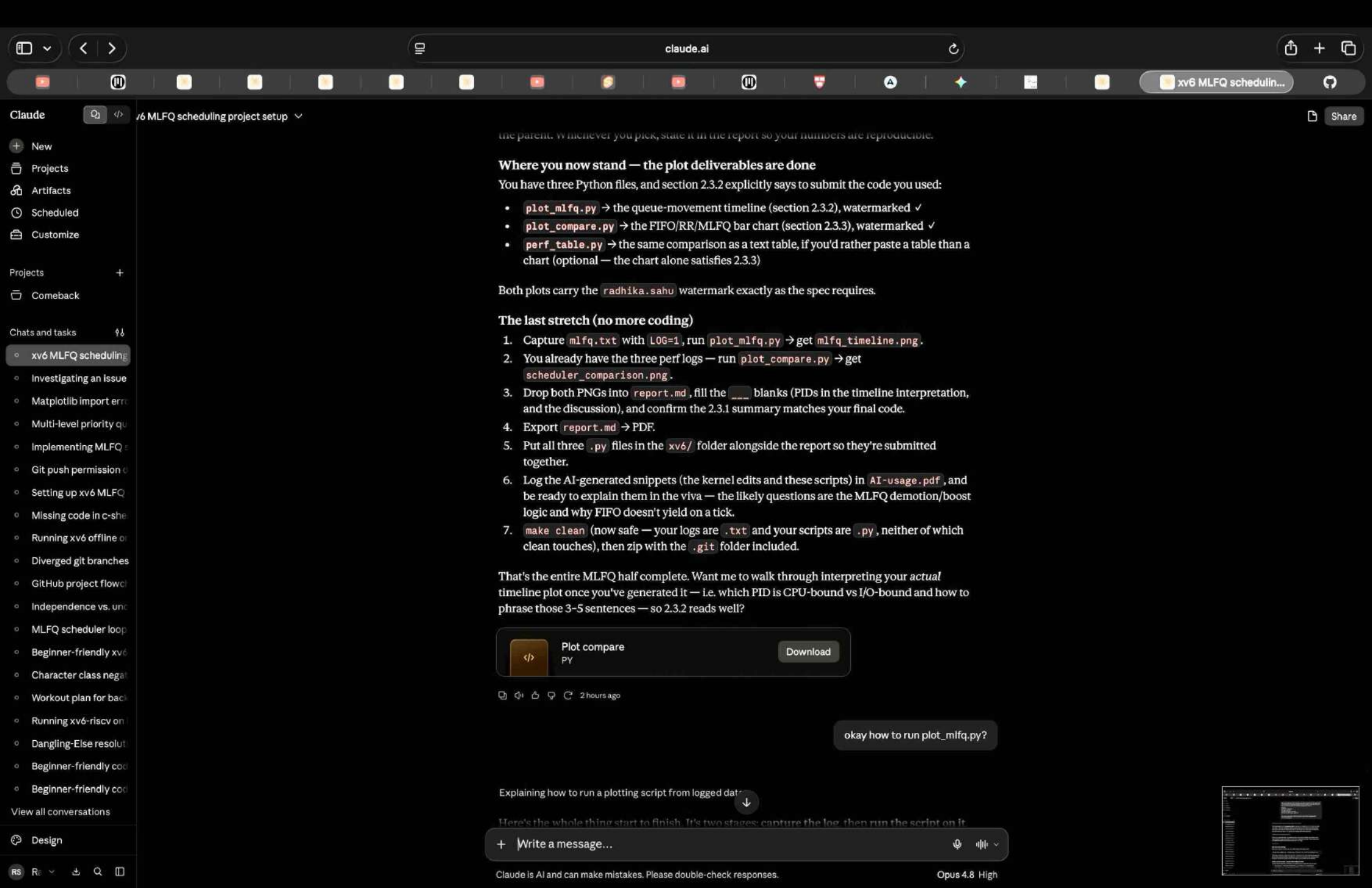
+ Write a message...

Claude is AI and can make mistakes. Please double-check responses.

Opus 4.8 High

xv6 MLFQ scheduling...





Opus 4.8 High

Open `kernel/proc.c`, find `m1fq_tick_yield()`, and locate this line (it's right after `p->ticks_used++`):

Delete it (or comment it out with `//`). Leave the other five `MLFQLOG(...)` lines — **NEW**, **DEMOTE**, **BOOST**, **SLEEP**, **WAKE** — exactly as they are. Those only fire on real transitions, so they won't flood.

```
make clean && make qemu SCHEDULER=MLFQ LOG=1 CPUS=1 | tee mlfq.txt
```

```
schedulertest
```

Check it captured:

A number in the dozens-to-hundreds is fine (it'll be far fewer than before, and that's correct).
Then:

If even after deleting the `TICK` line the boot is still too noisy to type, tell me — the next lever is to also drop the `SLEEP` / `WAKE` lines and keep only `NEW` / `REMOVE` / `POST`, which is the bare

2 hours ago

+ Write a message...  

Opus 4.8 High



Claude

New

Projects

Artifacts

Scheduled

Customize

Projects

Comeback

Chats and tasks

xv6 MLFQ scheduling

Investigating an issue

Matplotlib import error

Multi-level priority queue

Implementing MLFQ

Git push permission denied

Setting up xv6 MLFQ

Missing code in c-shell

Running xv6 offline on

Diverged git branches

GitHub project flowchart

Independence vs. un

MLFQ scheduler loop

Beginner-friendly xv6

Character class negat

Workout plan for back

Running xv6-riscv on

Dangling-Else resolut

Beginner-friendly cod

Beginner-friendly cod

View all conversations

Design

RS Re ↕ 🔍 📄

claude.ai

xv6 MLFQ schedulin...

Share

Wait for the `$` prompt, then type exactly:

```
schedulertest
```

Press Enter and watch the screen. You should see:

- `schedulertest: launching 3 children at tick ...`
- a burst of `[MLFQ] ... ev=NEW` and `ev=DEMOTE` lines as the children run and sink,
- finally `schedulertest: all children done at tick ...`.

Only *after* you see that last line, quit with `Ctrl-a` then `x`.

Verify before plotting

```
grep "ev=DEMOTE" mlfq.txt | head
```

If you see `DEMOTE` lines with `pid=3 / 4 / 5 / 6` in them, the workload is captured and the plot will show real queue movement. Then:

```
python3 plot_mlfq.py mlfq.txt
```

If you run `schedulertest`, watch it print "all children done," and the grep for `ev=DEMOTE` still comes back empty, then it's a genuine scheduler bug and we'll dig into `mlfq_tick_yield` together — paste me the first ~30 lines of `mlfq.txt` after the launch line and I'll spot it. But nine times out of ten this is just "the test needs to run before you quit," so try the recapture first.

I am just not able to type in scheduler test even after commenting MLFQLOG("[MLFQ] t=%d pid=%d q=%d ev=TICK\n", ticks, p->pid, p->priority);, should I comment all the MLFLOG?

Suspecting buffered output is hiding a ready prompt. >

Short answer: **no** — **don't comment out all of them**. If every `MLFQLOG` is gone, `mlfq.txt` will be empty and you'll have no plot at all. `TICK` was the only real flood; keep `NEW`, `DEMOTE`, and `BOOST` (those are exactly what the plot needs).

But here's the thing that's probably actually biting you, and it's not the logging.

Reason #1 (most likely): you didn't rebuild after commenting

+ Write a message...

Opus 4.8 High

Claude is AI and can make mistakes. Please double-check responses.

Editing `proc.c` changes nothing until you recompile. And the recompile only happens if `make clean` runs first. So the run you're testing may still be the *old* binary with `TICK` logging baked in. Always capture with the clean rebuild in one line:

If you commented the `TICK` line and then just ran `make qemu` (no clean), you'd see the exact "still flooding / can't type" behavior you're describing.

When you pipe `qemu` through `tee`, the `$` prompt often doesn't show up on screen (it's printed without a newline, so the pipe holds it back). It looks like there's nowhere to type — but the shell is sitting there ready. So try this: after boot settles, **just type `schedulertest` and press Enter even though you can't see what you're typing**. The keystrokes still reach xv6. Wait ~10 seconds, then quit with `Ctrl-a` then `x`.

If fighting the console is annoying, feed the command in automatically. This boots, waits, sends `schedulertest` on its own, lets the children run, and saves everything:

Let it sit for ~25 seconds, then press Ctrl-a then x (or Ctrl-c) to stop qemu. No typing required — the `echo` does it for you.

```
grep "ev=DEMOTE" mlfq.txt | head
```

```
python3 plot_mlfq.py mlfq.txt
```

So: keep **NEW / DEMOTE / BOOST**, rebuild with **make clean**, and use the hands-free line if typing won't cooperate. Try that and tell me what the **↓** shows — if PID 3/4/5/6 still never demote, then it's worth looking at **mifo tick yield** together.

Claude is AI and can make mistakes. Please double-check responses. Opus 4.8 High



Big update: your **current mlfq.txt** is actually good now — it has PIDs 3-6 with 13 demotes. The committed PNG is just stale (generated from the earlier bad capture). Let me regenerate the plot from your current log to confirm, and verify the kernel + perf logs.

Ran a command >

Kernel is fully patched (MLFQLOG defined, FIFO added, metrics printing, `rtime++` appears exactly once, response time set in all three schedulers). And the regenerated plot from your **current** log shows 6 processes moving. Let me look at it:

Viewing regenerated timeline from current mlfq.txt >

That plot is now correct and tells the real story: PID 4 (red) is CPU-bound — it sinks 0→1→2→3, then the 48-tick boost yanks it back to queue 0. PIDs 5/6 are mixed (settle around queue 1-2), PID 3 stays high, and after ~110 ticks only the shell (PID 2) remains, getting boosted every 48 ticks. **Re-commit this regenerated PNG** — the one in your repo is the stale version.

Let me verify your three perf logs actually produce a clean comparison:

Ran a command >

Your real comparison data is clean and tells the right story: FIFO has the worst response time (~20 ticks — the convoy effect), while RR and MLFQ respond almost instantly (~0.3). Let me regenerate the comparison chart from this real data too:

Ran a command ➤

Everything technical works. Now here's the honest gap analysis against the spec.

What's DONE

- **2.1 MLFQ engine [70]** — compiles and runs; all fixes confirmed in the repo (MLFQLOG defined, FIFO added, metrics printing, `rtime++` counted once, response time in all three schedulers). Your timeline plot proves the queues/demotion/boost all work.
- **2.2 comparison data** — `fifo/rr/mlfq_perf.txt` captured and parse cleanly.
- **2.3.2 timeline plot + python** — `plot_mlfq.py` present; log is good.
- **2.3.3 comparison chart + python** — `plot_compare.py` present.

What's MISSING X (this is what's left)

1. **The report itself** — `xv6/report.md` does not exist. This is the single biggest gap. Section 2.3 is 10 marks and it's the only place the grader sees your plots, your table, and your analysis. Right now none of that is written up.
2. **Stale committed PNG** — the `mlfq_time1.png` in your repo is the old broken one (only DFX 1.2). Make sure you reupload this correct `kernel.png` plot.

+ Write a message...

📄

🔍

🏠

📄

🔍

🏠

claude.ai

🔄

📄 + 📄

🔴

🟡

🟢

🟠

🟣

🟤

🟡

🟢

🟠

🟣

🟤

🔴

🟡

🟢

🟠

🟣

🟤

xv6 MLFQ schedulin...

🔄

Claude

🔍

📄

xv6 MLFQ scheduling project setup

📄

Share

+

New

📁

Projects

📄

Artifacts

🕒

Scheduled

🛠️

Customize

Projects

+

📁

Comeback

Chats and tasks

🔍

📄

xv6 MLFQ scheduling

🔍

Investigating an issue

🔍

Matplotlib import error

🔍

Multi-level priority queue

🔍

Implementing MLFQ scheduling

🔍

Git push permission error

🔍

Setting up xv6 MLFQ

🔍

Missing code in c-scheduler

🔍

Running xv6 offline on a Raspberry Pi

🔍

Diverged git branches

🔍

GitHub project flowchart

🔍

Independence vs. interdependence

🔍

MLFQ scheduler loop

🔍

Beginner-friendly xv6

🔍

Character class negation

🔍

Workout plan for back pain

🔍

Running xv6-riscv on a Raspberry Pi

🔍

Dangling-Else resolution

🔍

Beginner-friendly code

🔍

Beginner-friendly code

View all conversations

🎨

Design

📄

🔍

2.1 MLFQ engine [10] — compiles and runs; all fixes confirmed in the repo (MLFQLOG defined, FIFO added, metrics printing, `rttime++` counted once, response time in all three schedulers). Your timeline plot proves the queues/demotion/boost all work.

2.2 comparison data — `fifo/rr/mlfq_perf.txt` captured and parse cleanly.

2.3.2 timeline plot + python — `plot_mlfq.py` present; log is good.

2.3.3 comparison chart + python — `plot_compare.py` present.

What's MISSING ❌ (this is what's left)

1. The report itself — `xv6/report.md` does not exist. This is the single biggest gap. Section 2.3 is 10 marks and it's the only place the grader sees your plots, your table, and your analysis. Right now none of that is written up.

2. Stale committed PNG — the `mlfq_timeline.png` in your repo is the old broken one (only PID 1-2). Your current log produces the correct 6-process plot.

3. Re-commit `scheduler_comparison.png` from real data (I just regenerated it).

Since I now have your real numbers and a correct plot, I filled in the report with your actual data:

Created a file, shared files >

`report_final.md` is filled in with your real numbers and an accurate reading of your actual plot. I also handed back the two corrected PNGs (the timeline regenerated from your current log, and the comparison chart from real data).

Your to-do list to finish (all non-coding)

1. Rename `report_final.md` → `report.md` and put it in `xv6/`. Read it once and make sure the wording matches what you'll say in the viva — it's your report, not mine.

2. Replace the stale `mlfq_timeline.png` in `xv6/` with the regenerated one (the 6-process version).

3. Replace `scheduler_comparison.png` with the real-data one.

4. Convert `report.md` → PDF if you prefer (Markdown is allowed too — the spec says "PDF or Markdown").

5. Confirm `AI-usage.pdf` documents the kernel edits and the three Python scripts (it exists in your repo — just make sure the content is actually there).

6. `git add` the report + PNGs, commit, and push. Then `make clean` and zip with the `.git` folder for Moodle.

One thing to double-check before you submit

Your report table says the parent is excluded (`--children-only`). Make sure the PNG you commit was generated the same way, so the chart and table agree. Both my regenerations used `--children-only`, so if you commit the files I just gave you, they already match.

+ Write a message...

🔍 🔊

Claude is AI and can make mistakes. Please double-check responses.

Opus 4.8 High

Claude

New

Projects

Artifacts

Scheduled

Customize

Projects

Comeback

Chats and tasks

xv6 MLFQ scheduling

Investigating an issue

Matplotlib import error

Multi-level priority queue

Implementing MLFQ scheduler

Git push permission error

Setting up xv6 MLFQ

Missing code in c-shell

Running xv6 offline on Raspberry Pi

Diverged git branches

GitHub project flowchart

Independence vs. Unix

MLFQ scheduler loop

Beginner-friendly xv6

Character class negation

Workout plan for back pain

Running xv6-riscv on Raspberry Pi

Dangling-Else resolution

Beginner-friendly code

Beginner-friendly code

View all conversations

Design

RS Re ↕ 🔍 📄

6 MLFQ scheduling project setup

Share

Report final

Document · MD

Download

Mlfq timeline

Image · PNG

Download

Scheduler comparison

Image · PNG

Download

Download all

2 hours ago

okay give the final readme Aldo

Drafting a complete top-level README file.

Let me look at your current README so I extend it rather than clobber the C-shell half.

Ran a command

Your README already has a solid C-shell section but says nothing about the xv6/MLFQ half. I'll keep your C-shell prose exactly as-is and append a matching xv6 section, plus add MLFQ to the "completed" list at the top.

Ran a command, shared files

There's your final README. It keeps your entire C-shell write-up untouched, adds MLFQ to the completed-parts list at the top, and appends an xv6 section covering the build flags, how MLFQ works, the files you changed, and the exact commands to reproduce both plots.

To use it: rename `README_final.md` → `README.md` at your repo root (replacing the current one), then commit.

Where the whole MLFQ half stands now

Everything is done except committing the final artifacts. Your remaining checklist:

1. `report_final.md` → `xv6/report.md`

2. `README_final.md` → `README.md` (repo root)

3. Replace `xv6/mlfq_timeline.png` and `xv6/scheduler_comparison.png` with the

+ Write a message...

🔊 🔊 🔊

Claude is AI and can make mistakes. Please double-check responses.

Opus 4.8 High

Claude

+ New

Projects

Artifacts

Scheduled

Customize

Projects

Comeback

Chats and tasks

xv6 MLFQ scheduling

Investigating an issue

Matplotlib import error

Multi-level priority queue

Implementing MLFQ scheduling

Git push permission denied

Setting up xv6 MLFQ scheduling

Missing code in c-shell

Running xv6 offline on a Raspberry Pi

Diverged git branches

GitHub project flowchart

Independence vs. Unix

MLFQ scheduler loop

Beginner-friendly xv6

Character class negation

Workout plan for back pain

Running xv6-riscv on a Raspberry Pi

Dangling-Else resolution

Beginner-friendly code

Beginner-friendly code

View all conversations

Design

RS Re ↕ 🔍 📄

6 MLFQ scheduling project setup

Everything is done except committing the final artifacts. Your remaining checklist:

1. `report_final.md` → `xv6/report.md`

2. `README_final.md` → `README.md` (repo root)

3. Replace `xv6/mlfq_timeline.png` and `xv6/scheduler_comparison.png` with the regenerated versions

4. Confirm `AI-usage.pdf` documents the kernel edits + Python scripts

5. `git add -A` && `git commit` && `git push`, then `make clean` and zip with `.git` included

That's the full MLFQ half wrapped up — engine, FIFO/RR/MLFQ comparison, both watermarked plots, the report, and now the README. Good luck at the viva.

Readme final

Document · MD

Download

1. How to run the code

2. folder structure

3. design choices

4. assumptions if any

5. for xv6 you can include key changes you made

6. etc

1. How to run the code

2. folder structure

3. design choices

4. assumptions if any

5. for xv6 you can include key changes you made

6. etc

readme must have these

2 hours ago ↻ 🔗 📄

Drafting a README covering setup and design details for xv6.

Let me get your actual file layout so the folder-structure section is accurate, not guessed.

Ran 2 commands, read a file, shared files

This version is organized around exactly the six headings you asked for:

1. **How to run the code** — C-shell (`make all` → `shell.out`) and xv6 with all three scheduler flags plus the optional `LOG` / `PERF` flags.

2. **Folder structure** — a real tree of `c-shell/`, `c-shell/include`, and `xv6/`, matching your actual files with the `src/` / `lib/` / `fs/` / `kernel/` naming convention explained.

+ Write a message...

🗣️ 🔊 🔇

Claude is AI and can make mistakes. Please double-check responses.

Opus 4.8 High

xv6 MLFQ schedulin...

- 4. assumptions if any
- 5. for xv6 you can include key changes you made
- 6. etc
- 1. How to run the code
- 2. folder structure
- 3. design choices
- 4. assumptions if any
- 5. for xv6 you can include key changes you made
- 6. etc

readme must have these

Drafting a README covering setup and design details for xv6.

Let me get your actual file layout so the folder-structure section is accurate, not guessed.

Ran 2 commands, read a file, shared files >

This version is organized around exactly the six headings you asked for:

1. **How to run the code** — C-shell (`(make all → shell.out)`) and xv6 with all three scheduler flags plus the optional `LOG / PERF` flags.
2. **Folder structure** — a real tree of `c-shell/src`, `c-shell/include`, and `xv6/`, matching your actual files, with the `a_/b_/c_/d_/f_` naming convention explained.
3. **Design choices** — the xv6 reasoning (compile-time policy, `enter_seq` instead of real queues, split-out accounting, compile-gated logging), followed by your **entire original C-shell write-up** preserved as the C-shell design notes.
4. **Assumptions** — for both halves (1024-char input, first-group-only for `;/&`, deterministic frequency; and for xv6 the tick definition, `CPUS=1`, children-only averaging, 48-tick boost, 1/4/8/16 slices).
5. **Key xv6 changes** — a file-by-file list of what you modified and which rule each piece implements.
6. **Reproducing the report artifacts** — the exact capture + plot commands.

To use it: rename `README_final.md` → `README.md` at your repo root (replacing the current one), then commit. Nothing from your original C-shell notes was dropped — it's all still there under section 3.

That rounds out the whole submission: engine, comparison, plots, report, and a README that hits every required section. Anything you'd like me to tighten before you push?

Readme final
Document · MD

+ Write a message...

[illegible]